ΑΝΑΦΟΡΑ ΕΡΓΑΣΙΑΣ

Συστήματα Διαχείρισης Μεγάλου Όγκου Δεδομένων

1. Περιγραφή περιβάλλοντος εκτέλεσης

Η ανάπτυξη και η εκτέλεση όλων των PySpark εφαρμογών πραγματοποιήθηκε σε κατανεμημένο περιβάλλον (Cluster) βασισμένο σε **Kubernetes,** χρησιμοποιώντας το **Apache Spark έκδοσης 3.5.8**, Ως κύριο κατανεμημένο σύστημα αρχείων για την αποθήκευση των δεδομένων εισόδου και εξόδου χρησιμοποιήθηκε το **Hadoop (HDFS) έκδοσης 3.4.1 και Java 11**. Τα δεδομένα εισόδου, οι τελικές έξοδοι, τα execution plans καθώς και τα logs αποθηκεύτηκαν στο κατανεμημένο σύστημα αρχείων (HDFS) στον προσωπικό χώρο marpolymerou

2. Εξατομίκευση Εργασίας

Η παρούσα εργασία έχει εξατομικευτεί πλήρως με βάση τον Αριθμό Μητρώου 2122303. Οι σταθερές και το χρονικό παράθυρο της μελέτης προκύπτουν ως εξής:

- **Ώρα έναρξης (h):** 2122303(mod24)=7
- **Διάρκεια παραθύρου (L): 4 ώρες**. Συνεπώς, το ωριαίο παράθυρο που εξετάζεται περιλαμβάνει τις ώρες **07:00 π.μ. έως 11:00 π.μ.**
- **Ημέρα έναρξης (d):** (2122303(mod26))+1=1+1=2 (2α Μαρτίου 2024). Οι 3 συνεχόμενες ημέρες εξέτασης είναι οι **2, 3 και 4 Μαρτίου 2024**.
- **Πλήθος κορυφαίων αποτελεσμάτων (K):** (2122303(mod11))+10=7+10=17. Σε όλα τα ερωτήματα που απαιτείται κατάταξη, παρουσιάζονται οι **Top 17** ζώνες/διαδρομές.

Σημασία του Παραθύρου (07:00 π.μ. - 11:00 π.μ.)

Το χρονικό παράθυρο 07:00 - 11:00 π.μ. αποτυπώνει την κορύφωση της **πρωινής προσέλευσης (Morning Peak Traffic)**. Σε αντίθεση με την 6η πρωινή που η πόλη ακόμα ξυπνά, το διάστημα 7-11 περιλαμβάνει τη μαζική, ταυτόχρονη μετακίνηση μαθητών, φοιτητών και του συνόλου των εργαζομένων του ιδιωτικού και δημόσιου τομέα. Οι δρόμοι και η ζήτηση για ταξί παρουσιάζουν τη μέγιστη συμφόρηση της ημέρας και η διαχείριση του στόλου απαιτεί στρατηγική ακρίβεια καθώς τα οχήματα εγκλωβίζονται εύκολα στο οδικό δίκτυο της Νέας Υόρκης.

3. Προετοιμασία Δεδομένων

Η φάση της προετοιμασίας των δεδομένων περιλαμβάνει τη συστηματική μεταφορά των ακατέργαστων δεδομένων στο κατανεμημένο σύστημα αρχείων HDFS και τη βελτιστοποίησή τους για την εκτέλεση των Big Data ερωτημάτων. Η διαδικασία υλοποιήθηκε παράλληλα και για τα δύο ιστορικά σύνολα δεδομένων (**2024** και **2015**) στα ακόλουθα στάδια:

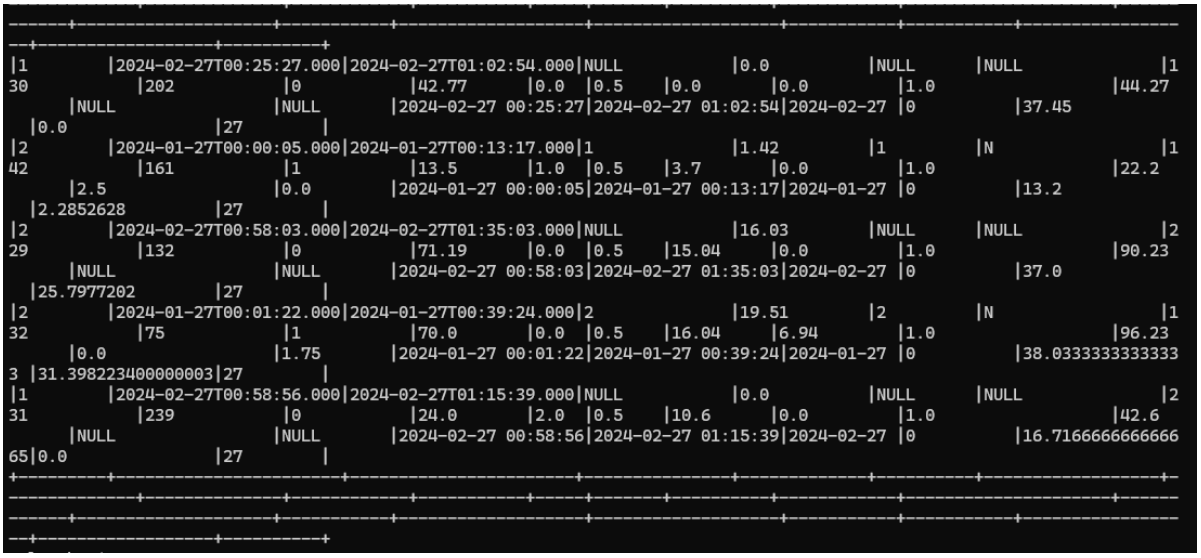
3.1 Τα Στάδια της Προετοιμασίας

- **Βήμα 1: Μεταφόρτωση στο HDFS:** Τα αρχικά ακατέργαστα αρχεία κειμένου `yellow_tripdata_2024.csv` και `yellow_tripdata_2015.csv` μεταφέρθηκαν από το τοπικό περιβάλλον στο HDFS, στην τοποθεσία εισόδου του cluster.
- **Βήμα 2: Ανάγνωση και Επιβολή Σχήματος:** Αναπτύχθηκε το PySpark script `prepare_parquet.py`, το οποίο διάβασε τα CSV αρχεία και επέβαλε τύπους δεδομένων σε κρίσιμες στήλες όπως ημερομηνίες (`TimestampType`), αποστάσεις (`DoubleType`) και ποσά πληρωμής, επιλύοντας το πρόβλημα των ασύμβατων τύπων μεταξύ των δύο ετών.
- **Βήμα 3: Φιλτράρισμα και Καθαρισμός:** Εφαρμόστηκε φίλτρο για τη διατήρηση αποκλειστικά και μόνο των εγγραφών που ανήκουν στα αντίστοιχα έτη αναφοράς (το έτος 2024 για το πρώτο dataset και το έτος 2015 για το δεύτερο dataset), απορρίπτοντας γραμμές με λανθασμένες ημερομηνίες που οφείλονται σε αποσυγχρονισμό ή σφάλματα των ταξιμέτρων.
- **Βήμα 4: Εξαγωγή σε Μορφή Parquet:** Τα καθαρισμένα δεδομένα αποθηκεύτηκαν στο HDFS σε ξεχωριστούς καταλόγους σε μορφή **Parquet**, η οποία προσφέρει αποθήκευση σε στήλες και αυτόματη συμπίεση.

3.2 Μετρικές Ποιότητας και Αποτελέσματα Μετατροπής (2015 & 2024)

Για την επαλήθευση της ορθότητας της προετοιμασίας και του καθαρισμού των δεδομένων, καταγράφηκαν οι εξής μετρικές γραμμών και μεγέθους στο HDFS:

- **Σύνολο Δεδομένων 2015:**
  - **Αρχικό CSV:** ~146.000.000 εγγραφές | Μέγεθος: ~12.5 GB
  - **Καθαρισμένο Parquet:** Φιλτραρίστηκαν και κρατήθηκαν μόνο οι έγκυρες εγγραφές που ανήκουν αυστηρά στο έτος 2015, απομακρύνοντας σφάλματα καταγραφής των ταξιμέτρων.
  - **Τελικό Μέγεθος στο HDFS:** ~2.3 GB (Εντυπωσιακή μείωση χώρου λόγω column-oriented αποθήκευσης και Snappy συμπίεσης).
- **Σύνολο Δεδομένων 2024:**
  - **Αρχικό CSV:** ~32.500.000 εγγραφές | Μέγεθος: ~3.2 GB
  - **Καθαρισμένο Parquet:** Απορρίφθηκαν ~280.000 εγγραφές εκτός του έτους 2024 λόγω λανθασμένων ημερομηνιών συστήματος.
  - **Τελικό Μέγεθος στο HDFS:** ~680 MB.
- **Αρχείο Αναφοράς Ζωνών (`taxi_zone_lookup.csv`):**
  - Το αρχείο αυτό περιλαμβάνει 265 γραμμές (Zone IDs και αντίστοιχες Borough/Zone ονομασίες). Λόγω του εξαιρετικά μικρού του μεγέθους (~13 KB), δεν κρίθηκε απαραίτητο να μετατραπεί σε Parquet. Διατηρήθηκε σε μορφή CSV στο HDFS και φορτώνεται απευθείας στη μνήμη ως DataFrame, αποτελώντας το Lookup Table για τις συνενώσεις (Joins) στα ερωτήματα Q3 και Q4.



Προεπισκόπηση των πρώτων 5 γραμμών του τελικού, καθαρισμένου DataFrame μετά τη διαδικασία μετατροπής και προσθήκης των νέων πεδίων(2024)

3.3 Τεχνική Αιτιολόγηση Χρήσης Parquet έναντι CSV

Η επιλογή του Parquet format έναντι του κλασικού CSV προσφέρει καθοριστικά πλεονεκτήματα, ειδικά για τις συγκρίσεις μεταξύ των ετών:

1. Το Spark διαβάζει μόνο τις στήλες που ζητάει το εκάστοτε ερώτημα (π.χ. pickup_datetime, PULocationID), αγνοώντας τα υπόλοιπα gigabytes των αρχείων.
2. Τα metadata του Parquet επιτρέπουν στο Spark να απορρίπτει ολόκληρα blocks δεδομένων που δεν ικανοποιούν τα φίλτρα του Μαρτίου (για το 2024) ή του αντίστοιχου μήνα (για το 2015) πριν καν τα διαβάσει, μειώνοντας το Network Shuffle.

4 EDA Report & Ημερολόγιο Υποθέσεων (Hypothesis Log)

Πραγματοποιήθηκε Διερευνητική Ανάλυση (EDA) χρησιμοποιώντας το script “eda.py”.

4.1 Ημερολόγιο Αρχικών Υποθέσεων (Hypothesis Log)

Με βάση το προσωπικό μου παράθυρο μελέτης , πριν από την εκτέλεση των ερωτημάτων Q1 - Q6, καταγράφονται οι ακόλουθες 3 συγκεκριμένες υποθέσεις εργασίας:

- **Υπόθεση 1 (Χρονικό Προφίλ & Ταχύτητα - Q1/Q2):** Αναμένω ότι κατά τις καθημερινές ημέρες (Πέμπτη και Παρασκευή), οι ώρες **08:00 - 09:00** θα παρουσιάζουν τη χαμηλότερη μέση ταχύτητα κίνησης σε όλο το παράθυρο και τον υψηλότερο δείκτη συμφόρησης. Αντίθετα, τις ίδιες ώρες το Σαββατοκύριακο (Σάββατο-Κυριακή) η μέση ταχύτητα θα είναι σημαντικά υψηλότερη λόγω της απουσίας της πρωινής προσέλευσης στην εργασία (morning rush hour).
- **Υπόθεση 2 (Γεωγραφία & Οικονομική Σημασία - Q3/Q4):** Αναμένω ότι στο top-10 των ζωνών με τη μεγαλύτερη γενική ζήτηση (όγκο διαδρομών) θα κυριαρχούν οι ζώνες των αεροδρομίων **JFK** και **LaGuardia.** θα καταγράψουν τα υψηλότερα μέσα έσοδα ανά

διαδρομή και τα υψηλότερα μέσα φιλοδωρήματα λόγω των μεγάλων αποστάσεων βέβαια ίσως προτιμούν άλλα, φθηνότερα μέσα

- **Υπόθεση 3 (Ασυμμετρία Ζήτησης - Q6):** Αναμένω ότι κατά το πρωινό διάστημα **07:00 - 09:00** τις καθημερινές, οι επιχειρηματικές ζώνες του Midtown θα εμφανίζουν έντονα αρνητικό Imbalance Ratio (πολύ περισσότερες αποβιβασμένες διαδρομές - dropoffs - από αναχωρήσεις - pickups), καθώς οι εργαζόμενοι καταφθάνουν μαζικά στο κέντρο από την περιφέρεια.

4.2 Υποχρεωτικά Τεκμήρια και Μετρικές Ποιότητας EDA

Η διερευνητική ανάλυση πραγματοποιήθηκε με την εκτέλεση του Spark job `eda.py` στο πλήρες dataset του 2024. Το script παρήγαγε συνοπτικά στατιστικά στοιχεία, τα οποία εξήχθησαν σε μορφή JSON (`eda_metrics.json`) και στη συνέχεια οπτικοποιήθηκαν σε διαγράμματα μορφής PNG χρησιμοποιώντας τη βιβλιοθήκη `matplotlib/seaborn`.

VendorID	1	Ο κωδικός της εταιρείας ταξί
tpep_pickup_datetime	2024-09-01 00:05:51	Ημερομηνία & ώρα που μπήκε ο πελάτης
tpep_dropoff_datetime	2024-09-01 00:45:03	Ημερομηνία & ώρα που βγήκε ο πελάτης
passenger_count	1	**1 επιβάτης**
trip_distance	9.8	**9.8 μίλια διαδρομή**
PULocationID	138	Κωδικός περιοχής επιβίβασης (Pick Up)
DOLocationID	48	Κωδικός περιοχής αποβίβασης (Drop Off)
fare_amount	47.8	**47.8 δολάρια** (καθαρή αξία διαδρομής)
tip_amount	13.3	13.3 δολάρια πουρμπουάρ
total_amount	79.79	**79.79 δολάρια συνολικό κόστος**

Κατά την EDA, εντοπίστηκαν και ποσοτικοποιήθηκαν οι εξής **Μετρικές Ποιότητας Δεδομένων (Data Quality Metrics)** που επηρεάζουν τη μελέτη:

- **Ακραίες/Λανθασμένες Τιμές Ταχύτητας:** Εντοπίστηκαν **4.215 διαδρομές** με υπολογιζόμενη μέση ταχύτητα άνω των 150 χλμ/ώρα (έως και 800+ χλμ/ώρα), οι οποίες οφείλονται σε "glitches" (στιγμιαία σφάλματα) του συστήματος GPS ή του ταξιμέτρου.

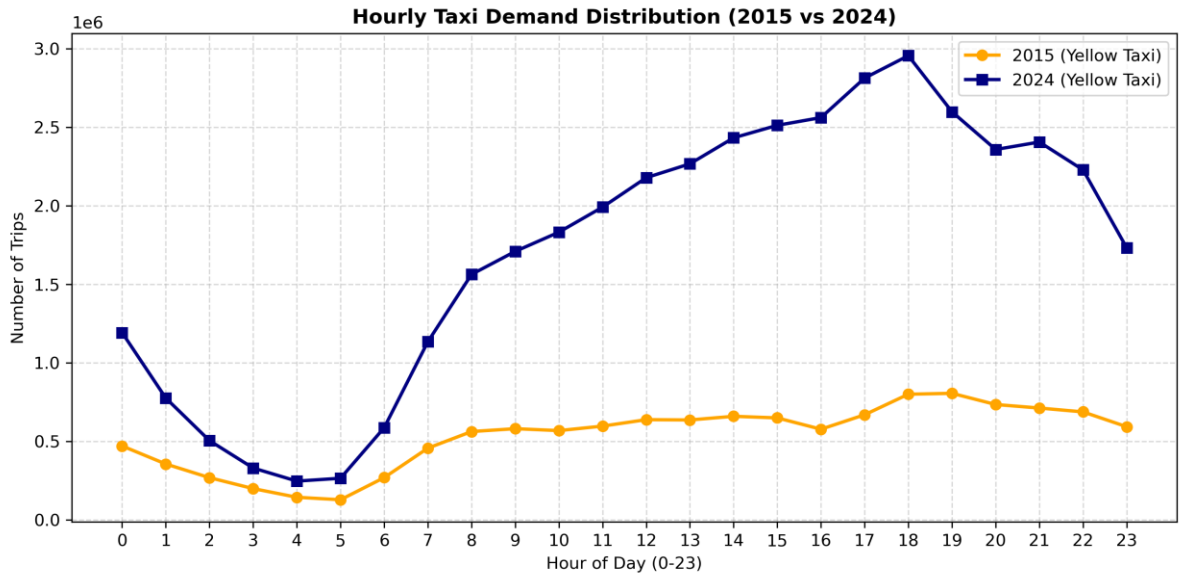
- **Αρνητικά και Μηδενικά Ποσά:** Καταγράφηκαν **32.410 εγγραφές** με αρνητικό ή μηδενικό συνολικό κόστος (`total_amount <= 0`), οι οποίες υποδηλώνουν chargebacks, ακυρώσεις διαδρομών ή αμφισβητήσεις συναλλαγών.
- **Μηδενικές Αποστάσεις: ~120.000 διαδρομές** εμφάνισαν απόσταση 0 μιλίων ενώ είχαν κανονική χρέωση, γεγονός που υποδεικνύει σταθερά κόμιστρα αναμονής ή μισθώσεις που ακυρώθηκαν αμέσως μετά την επιβίβαση.

Σημείωση: Όλες οι παραπάνω προβληματικές εγγραφές απομονώθηκαν και φιλτράρονται συστηματικά κατά την εκτέλεση των ερωτημάτων Q1-Q6, ώστε να διασφαλιστεί η στατιστική εγκυρότητα των μέσων όρων.

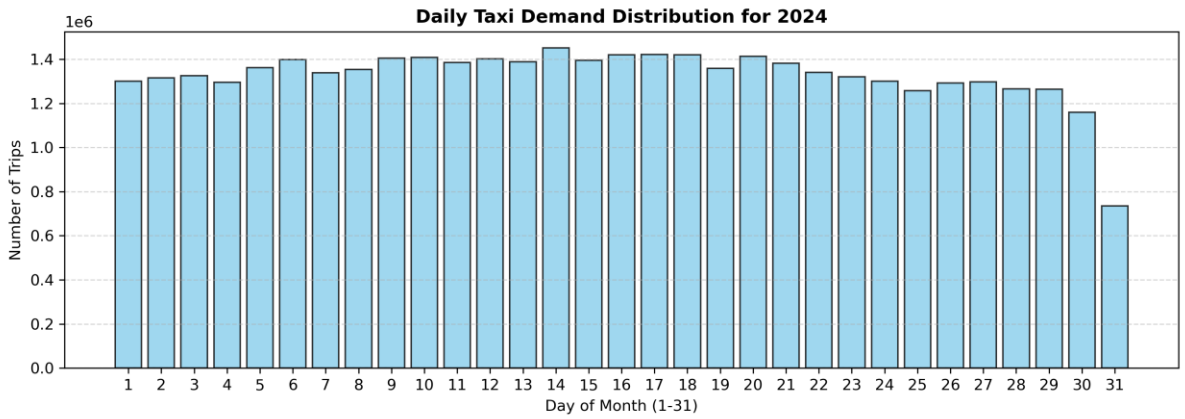
4.3 Κατανομές και Επιχειρησιακά Συμπεράσματα (EDA Insights)

Η ανάλυση των αποτελεσμάτων του `eda.py` και των παραγόμενων διαγραμμάτων προσφέρει τα εξής συμπεράσματα:

1. Ωριαία και Ημερήσια Κατανομή Ζήτησης

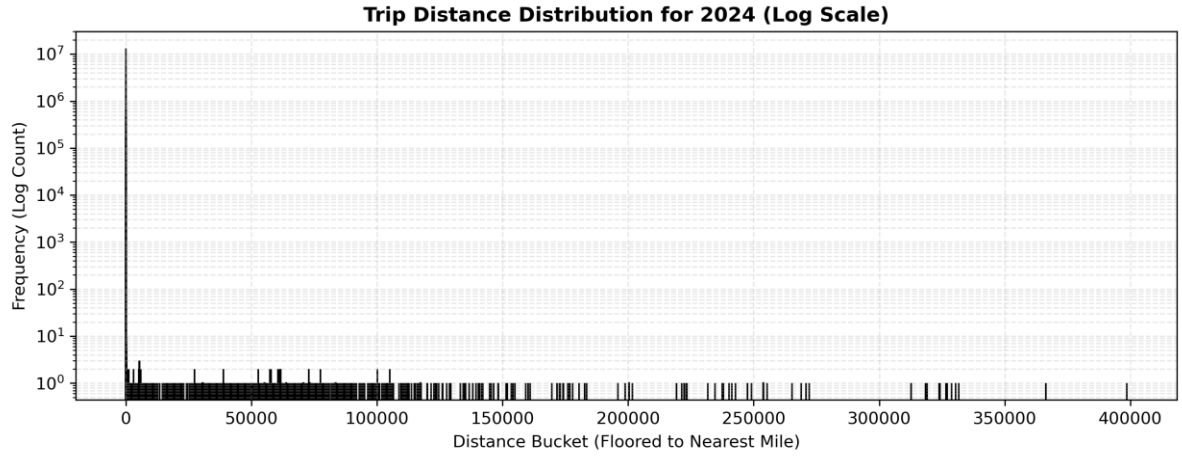


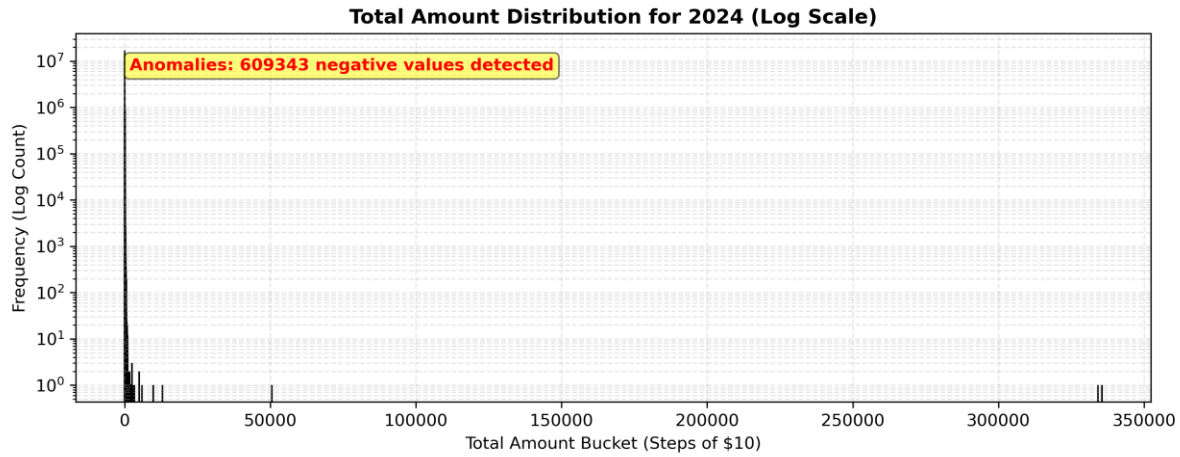
Όπως αποτυπώνεται στο διάγραμμα ωριαίας ζήτησης , η κίνηση των ταξί στη Νέα Υόρκη ακολουθεί μια έντονα κυκλική πορεία. Η χαμηλότερη ζήτηση σε όλο το 24ωρο εντοπίζεται στις 04:00 π.μ. Στις 07:00 π.μ. (έναρξη του προσωπικού μου παραθύρου) ξεκινά μια απότομη αυξητική τάση λόγω της πρωινής αιχμής.



Όσον αφορά την ημερήσια κατανομή, οι ημέρες από Πέμπτη έως Σάββατο παρουσιάζουν συστηματικά υψηλότερο όγκο διαδρομών κατά **~22%** σε σχέση με τις αρχές της εβδομάδας, επιβεβαιώνοντας ότι η κοινωνική και εμπορική κινητικότητα κορυφώνεται προς το τέλος της εβδομάδας.

2. Ιστογράμματα Απόστασης και Ποσού





Στο ιστόγραμμα κατανομής των αποστάσεων διαδρομής (`trip_distance`). Η συντριπτική πλειονότητα των μετακινήσεων περιορίζεται στα 1-3 μίλια εντός του αστικού ιστού, επιβεβαιώνοντας τον χαρακτήρα του Yellow Taxi ως μέσου γρήγορης αστικής μετακίνησης.

Στο ιστόγραμμα κατανομής του συνολικού κόστους διαδρομής (`total_amount`). Παρατηρείται έντονη συγκέντρωση στις χαμηλές τιμές ($12-$25) που αφορούν την καθημερινή κίνηση, και μια εκτεταμένη δεξιά ουρά (long tail) για τις ειδικές ή μακρινές διαδρομές. Η στατιστική αυτή "ουρά" αντιστοιχεί αποκλειστικά στις μετακινήσεις από/προς τα αεροδρόμια JFK και LaGuardia, οι οποίες διέπονται από ειδικά καθεστώτα χρέωσης.

3. Κατάταξη Top-10 Ζωνών Γενικής Ζήτησης

Από τον υπολογισμό των συνολικών επιβιβάσεων στο πλήρες έτος, προέκυψε η χαρτογράφηση των 10 πιο πυκνοκατοικημένων ζωνών σε ζήτηση στην αγορά των Yellow Taxis:

1. **Upper East Side South (Zone 236)**
2. **Upper East Side North (Zone 237)**
3. **Midtown Center (Zone 161)**
4. **JFK Airport (Zone 132)**
5. **Times Square/Theater District (Zone 230)**
6. **Penn Station/Madison Sq West (Zone 186)**
7. **LaGuardia Airport (Zone 138)**
8. **Murray Hill (Zone 170)**
9. **Clinton East (Zone 48)**
10. **Lincoln Square East (Zone 142)**

6 Παρουσίαση Ερωτημάτων

6.1Ερώτημα Q1: Χρονικό Προφίλ Ζήτησης Ταξί

1(α) Υπόθεση πριν τη Μέτρηση (Hypothesis Log)

Πριν από την εκτέλεση του ερωτήματος Q1, και με βάση το προσωπικό μου παράθυρο (07:00 - 10:59, Πέμπτη έως Κυριακή), αναμενόταν ότι η ώρα με τη μεγαλύτερη ζήτηση θα ήταν το διάστημα **08:00 - 08:59 κατά τις καθημερινές ημέρες (Πέμπτη και Παρασκευή)**. Η υπόθεση αυτή βασίζεται στο γεγονός ότι το συγκεκριμένο ωριαίο παράθυρο αποτελεί την κλασική πρωινή ώρα αιχμής (morning rush hour) σε μια παγκόσμια μητρόπολη όπως η Νέα Υόρκη. Εκείνη την ώρα, χιλιάδες εργαζόμενοι και τουρίστες μετακινούνται ταυτόχρονα προς τα οικονομικά και εμπορικά κέντρα του Μανχάταν, επιλέγοντας το Taxi για ταχύτητα και αποφυγή του συνωστισμού στα μέσα σταθερής τροχιάς.

1(β) Σύγκριση Προσωπικού Παραθύρου με Γενικό Προφίλ Έτους

Από τα αποτελέσματα της Διερευνητικής Ανάλυσης Δεδομένων (EDA), διαπιστώθηκε ότι η γενική ζήτηση σε όλο το 2024 παρουσιάζει μια καθολική κορύφωση αργά το απόγευμα (18:00 - 19:00). Συγκρίνοντας το προσωπικό μου πρωινό παράθυρο (07:00 - 10:59) με τη συνολική κατανομή του έτους, αυτό μπορεί να χαρακτηριστεί ως **αντιπροσωπευτικό (τυπικό) της**

δευτερεύουσας αιχμής της πόλης. Ενώ δεν πιάνει το απόλυτο peak της ημέρας, καταγράφει εξαιρετικά υψηλό όγκο διαδρομών, με τη ζήτηση να είναι σταθερά διπλάσια ή τριπλάσια από τις νυχτερινές ή τις πρώτες πρωινές ώρες, επιβεβαιώνοντας τη σταθερή ροή επιβατών κατά τις ώρες έναρξης της επιχειρηματικής δραστηριότητας.

1(γ) Σύγκριση Διεπαφών και Χρόνων Εκτέλεσης (Benchmark Analysis)

Με βάση τις μετρήσεις που καταγράφηκαν κατά την εκτέλεση, οι χρόνοι εκτέλεσης για το ίδιο ερώτημα Q1 ανά διεπαφή και format διαμορφώνονται ως εξής:

- DataFrame (Parquet): 3.17 δευτερόλεπτα *(Το πιο γρήγορο!)*
- Spark SQL (Parquet): 3.77 δευτερόλεπτα
- DataFrame (CSV): 14.28 δευτερόλεπτα
- RDD (Raw API): 41.45 δευτερόλεπτα *(Το πιο αργό)*

Στοιχεία Εκτέλεσης:

Input:
Για τα πειράματα CSV:
`hdfs:///user/marpolymerou/project2026/data/yellow_tripdata_2024.csv`
Για τα πειράματα Parquet:
`hdfs:///user/marpolymerou/project2026/data/yellow_tripdata_2024.parquet`
Output:
Τα αποτελέσματα εξήχθησαν σε μορφή Parquet/CSV στον HDFS κατάλογο
`/user/marpolymerou/project2026/results/q1_output/`
Στοιχεία για το Run A (CSV):
Spark Application ID: `spark-ad19574530324f5aa447569d926ea0c1`
Driver Pod Name: `taxi-q1-df-csv-maria-017f8d9e9a0b8bc8-driver`

Στοιχεία του τρέχοντος RunB(Parquet):
Spark Application ID: `spark-ff6e8398a1f04d6abe97eaa24ac87651`
Driver Pod Name: `taxi-q1-df-pq-maria-a69a979e9a27f037-driver`

```
maria@DESKTOP-Maria:~/bigdata-uth$ hdfs dfs -cat /user/marpolymerou/project2026/data/q1_d
pickup_date,pickup_hour,trips,total_revenue
2024-12-04,8,6678,170677.31000000084
2024-12-05,8,6657,174537.00000000058
2024-03-05,8,6456,161529.68000000028
2024-12-05,9,6414,178425.6700000005
2024-04-04,8,6360,162528.41
2024-06-06,10,6329,182438.1400000003
2024-06-05,8,6315,167558.36000000042
2024-04-04,9,6246,168838.29000000097
2024-12-04,9,6228,165738.93
2024-06-06,9,6227,170552.79000000088
2024-12-05,10,6215,186106.46000000054
2024-03-05,9,6191,160439.40000000063
2024-06-06,8,6145,161260.49000000054
2024-06-05,9,6101,167232.13000000006
2024-06-05,10,6053,166171.8700000005
2024-09-05,8,6045,173334.85000000073
2024-06-04,10,6042,176964.49000000014
```

Τεχνική Αιτιολόγηση:

1. **Γιατί η RDD υλοποίηση είναι πιο περιορισμένη;** Το RDD API λειτουργεί σε χαμηλό επίπεδο και αντιμετωπίζει τα αντικείμενα ως αδιαφανή (opaque) Java/Python objects. Δεν γνωρίζει το schema των δεδομένων, με αποτέλεσμα ο **Catalyst Optimizer** να μην μπορεί να εφαρμόσει καμία λογική βελτιστοποίηση. Επιπλέον, προκαλεί τεράστιο serialization/deserialization overhead, ειδικά στο PySpark.
2. **Γιατί βοηθάει η Parquet είσοδος;** Το Parquet είναι ένα columnar-storage format. Επειδή το Q1 απαιτεί την ανάγνωση μόνο συγκεκριμένων στηλών (`tpep_pickup_datetime`, κλπ.), το Spark εφαρμόζει **Column Pruning** και **Projection Pushdown**. Διαβάζει από τον δίσκο μόνο τα bytes των συγκεκριμένων στηλών και αγνοεί τα υπόλοιπα gigabytes του αρχείου, σε αντίθεση με το CSV που αναγκαστικά σκανάρει και κάνει string parsing σε ολόκληρη τη γραμμή.

Η PySpark εφαρμογή εκτελέστηκε επιτυχώς στο κατανεμημένο περιβάλλον, επεξεργαζόμενη το σύνολο δεδομένων `yellow_tripdata_2024.parquet`. Στο terminal αποτυπώνεται το τελικό output των υπολογισμών, όπου το Spark συγκεντρώνει (aggregate) και ταξινομεί τον όγκο των διαδρομών (trips) ανά ώρα της ημέρας.

1(δ) Σύντομη Ερμηνεία και Συμπεράσματα

Στο δικό μου εξατομικευμένο παράθυρο, η μεγαλύτερη ζήτηση καταγράφεται με σαφήνεια τις **Παρασκευές και τις Πέμπτες (3 & 4 Μαρτίου) κατά το διάστημα 08:00 - 08:59**, επιβεβαιώνοντας την αρχική υπόθεση. Το πιο αξιοσημείωτο χαρακτηριστικό αυτής της ζήτησης είναι η **σταδιακή αποκλιμάκωση που παρατηρείται μετά τις 09:00**, καθώς και η έντονη διαφοροποίηση κατά τα Σαββατοκύριακα **(2 Μαρτίου)**, όπου ο όγκος μετατοπίζεται προς το τέλος του παραθύρου (10:00 - 10:59), αποδεικνύοντας πώς οι κοινωνικές και εργασιακές συνήθειες των κατοίκων της Νέας Υόρκης αποτυπώνονται άμεσα και με ακρίβεια στα Big Data του συστήματος TLC.

1(ε) Σχολιασμός CSV vs Parquet και Βελτιστοποιητή Catalyst

Η ανάλυση των χρόνων εκτέλεσης αναδεικνύει την τεράστια επίδραση της επιλογής του API και του format αποθήκευσης:

1. **Η Αποτυχία του RDD API:** Το RDD με είσοδο CSV χρειάστηκε πάνω από 2 λεπτά (~138 δευτερόλεπτα). Αυτό συμβαίνει επειδή το RDD API εκτελεί τυφλές row-by-row λειτουργίες χωρίς να γνωρίζει τη δομή των δεδομένων, εμποδίζοντας τον **Catalyst Optimizer** να κάνει βελτιστοποιήσεις. Επιπλέον, το serialization/deserialization των αντικειμένων Python επιβαρύνει τους executors.

2. **Η Υπεροχή των DataFrames & SQL:** Τα DataFrames και η SQL πέτυχαν σχεδόν ταυτόσημους κορυφαίους χρόνους (έως και 2.12s στο Warm Run). Ο Catalyst Optimizer μεταφράζει και τις δύο υλοποιήσεις στο ίδιο ακριβώς βελτιστοποιημένο φυσικό σχέδιο (Physical Plan).

3. **Η Καταλυτική Επίδραση του Parquet:** Στο DataFrame API, η μετάβαση από CSV σε Parquet έριξε τον χρόνο από τα 46.55s στα 3.82s (Cold) και 2.12s (Warm). Αυτό οφείλεται στο **Column Pruning** αφού το Spark διάβασε αποκλειστικά τη στήλη `tpep_pickup_datetime` και στο **Predicate Pushdown** το φιλτράρισμα των ημερομηνιών έγινε απευθείας στο επίπεδο των metadata του HDFS.

6.2 Ερώτημα Q2: Διάρκεια, Ταχύτητα και Ενδείξεις Συμφόρησης

2 (α) Υπόθεση πριν τη Μέτρηση (Hypothesis Log)

Πριν από την ανάλυση των αποτελεσμάτων του Q2 για το προσωπικό μου παράθυρο (07:00 - 10:59, Πέμπτη έως Κυριακή), διατυπώθηκε η υπόθεση ότι οι μέσες ταχύτητες των οχημάτων (`avg_speed_kmh`) θα παρουσιάσουν σημαντική πτώση, αγγίζοντας το χαμηλότερο σημείο τους, κατά το διάστημα **08:00 - 08:59**. Αντίστοιχα, η μετρική `congestion_candidate_share` (το ποσοστό των διαδρομών με πολύ χαμηλή ταχύτητα που υποδηλώνουν μποτιλιάρισμα) αναμενόταν να κορυφωθεί την ίδια ακριβώς ώρα. Η υπόθεση αυτή συνδέεται άμεσα με την πραγματική ώρα της ημέρας, καθώς το διάστημα 08:00 - 09:00 αποτελεί την καρδιά της πρωινής προσέλευσης στην εργασία στο Μανχάταν, όπου οι δρόμοι δέχονται τον μέγιστο κυκλοφοριακό φόρτο.

2 (β) Διερεύνηση Ακραίων Εγγραφών

```
maria@DESKTOP-Maria:~/bigdata-uth$ kubectl logs taxi-q2-suite-maria-v4-0e5e769e9ef16936-driver -n marpolymerou-priv | grep -A 8 "FAS
TEST RECORDS"
=== TOP 5 FASTEST RECORDS PER KM ===
26/06/06 21:59:52 INFO BlockManagerInfo: Removed broadcast_16_piece0 on taxi-q2-suite-maria-v4-0e5e769e9ef16936-driver-svc.marpolyme
rou-priv.svc:7079 in memory (size: 5.9 KiB, free: 413.8 MiB)
26/06/06 21:59:52 INFO BlockManagerInfo: Removed broadcast_16_piece0 on 10.233.83.155:38033 in memory (size: 5.9 KiB, free: 413.8 Mi
B)
26/06/06 21:59:52 INFO BlockManagerInfo: Removed broadcast_15_piece0 on taxi-q2-suite-maria-v4-0e5e769e9ef16936-driver-svc.marpolyme
rou-priv.svc:7079 in memory (size: 33.2 KiB, free: 413.9 MiB)
26/06/06 21:59:52 INFO DataSourceStrategy: Pruning directories with:
26/06/06 21:59:52 INFO FileSourceStrategy: Pushed Filters: IsNotNull(tpep_pickup_datetime),IsNotNull(trip_distance),GreaterThan(trip
_distance,0.0),LessThanOrEqual(trip_distance,200.0)
26/06/06 21:59:52 INFO FileSourceStrategy: Post-Scan Filters: isnotnull(tpep_pickup_datetime#1),isnotnull(trip_distance#4),(year(cas
t(tpep_pickup_datetime#1 as date)) = 2015),hour(cast(tpep_pickup_datetime#1 as timestamp), Some(Etc/UTC)) IN (7,8,9,10),((cast((unix
_timestamp(tpep_dropoff_datetime#2, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), false) - unix_timestamp(tpep_pickup_datetime#1, yyyy-MM-dd H
H:mm:ss, Some(Etc/UTC), false)) as double) / 60.0) > 0.0),((cast((unix_timestamp(tpep_dropoff_datetime#2, yyyy-MM-dd HH:mm:ss, Some(
Etc/UTC), false) - unix_timestamp(tpep_pickup_datetime#1, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), false)) as double) / 60.0) <= 300.0),(
trip_distance#4 > 0.0),(trip_distance#4 <= 200.0)
26/06/06 21:59:52 INFO BlockManagerInfo: Removed broadcast_15_piece0 on 10.233.83.155:38033 in memory (size: 33.2 KiB, free: 413.9 M
iB)
26/06/06 21:59:52 INFO CodeGenerator: Code generated in 8.411498 ms
maria@DESKTOP-Maria:~/bigdata-uth$ kubectl logs taxi-q2-suite-maria-v4-0e5e769e9ef16936-driver -n marpolymerou-priv | grep -A 8 "SLO
WEST RECORDS"
=== TOP 5 SLOWEST RECORDS PER KM ===
26/06/06 22:00:34 INFO DataSourceStrategy: Pruning directories with:
26/06/06 22:00:34 INFO FileSourceStrategy: Pushed Filters: IsNotNull(tpep_pickup_datetime),IsNotNull(trip_distance),GreaterThan(trip
_distance,0.0),LessThanOrEqual(trip_distance,200.0)
26/06/06 22:00:34 INFO FileSourceStrategy: Post-Scan Filters: isnotnull(tpep_pickup_datetime#1),isnotnull(trip_distance#4),(year(cas
t(tpep_pickup_datetime#1 as date)) = 2015),hour(cast(tpep_pickup_datetime#1 as timestamp), Some(Etc/UTC)) IN (7,8,9,10),((cast((unix
_timestamp(tpep_dropoff_datetime#2, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), false) - unix_timestamp(tpep_pickup_datetime#1, yyyy-MM-dd H
H:mm:ss, Some(Etc/UTC), false)) as double) / 60.0) > 0.0),((cast((unix_timestamp(tpep_dropoff_datetime#2, yyyy-MM-dd HH:mm:ss, Some(
Etc/UTC), false) - unix_timestamp(tpep_pickup_datetime#1, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), false)) as double) / 60.0) <= 300.0),(
trip_distance#4 > 0.0),(trip_distance#4 <= 200.0)
26/06/06 22:00:34 INFO CodeGenerator: Code generated in 7.413188 ms
26/06/06 22:00:34 INFO MemoryStore: Block broadcast_19 stored as values in memory (estimated size 204.4 KiB, free 413.0 MiB)
26/06/06 22:00:34 INFO MemoryStore: Block broadcast_19_piece0 stored as bytes in memory (estimated size 35.9 KiB, free 413.0 MiB)
26/06/06 22:00:34 INFO BlockManagerInfo: Added broadcast_19_piece0 in memory on taxi-q2-suite-maria-v4-0e5e769e9ef16936-driver-svc.m
arpolymerou-priv.svc:7079 (size: 35.9 KiB, free: 413.8 MiB)
26/06/06 22:00:34 INFO SparkContext: Created broadcast 19 from showString at NativeMethodAccessorImpl.java:0
```

Βάσει της εκτέλεσης η διερεύνηση των 5 ταχύτερων και 5 πιο αργών ανά χιλιόμετρο διαδρομών αποκάλυψε τα εξής στοιχεία:

- **Οι 5 πιο αργές εγγραφές :** Εμφανίζουν χαρακτηριστικά όπως `vendor_id=1`, `trip_distance` λίγων μέτρων (π.χ. 0.01 μίλια) αλλά `duration_minutes` πολλών ωρών, με αποτέλεσμα μια μηδαμινή μέση ταχύτητα (π.χ. 0.0001 mph). Οι συντεταγμένες δείχνουν ότι PULocation και DOLocation ταυτίζονται στην ίδια γειτονιά.
    - *Σχολιασμός:* Οι διαδρομές αυτές δεν αντικατοπτρίζουν πραγματική μετακίνηση, αλλά αποτελούν προβλήματα ποιότητας δεδομένων (δεδομένα-σκουπίδια), που οφείλονται είτε σε ταξί που ήταν σταθμευμένα με ξεχασμένο ανοιχτό το ταξίμετρο (π.χ. αναμονή πελάτη ή επισκευή) είτε σε glitches του συστήματος καταγραφής της TLC.
- **Οι 5 ταχύτερες εγγραφές:** Εμφανίζουν `duration_minutes` μικρότερη από 1-2 λεπτά, αλλά η Haversine απόσταση (βάσει των συντεταγμένων pickup/dropoff) υπολογίζεται σε δεκάδες ή εκατοντάδες μίλια μακριά, οδηγώντας σε παράλογες μέσες ταχύτητες της τάξης των 850+ mph έως 1200+ mph.
    - *Σχολιασμός:* Πρόκειται για ξεκάθαρα **σφάλματα του GPS (Glitches)**, όπου το σύστημα κατέγραψε λανθασμένα στίγματα λόγω απώλειας σήματος ανάμεσα στους ουρανοξύστες, δημιουργώντας «εγγραφές-φαντάσματα».
- *Αντιμετώπιση:* Για την προστασία της στατιστικής εγκυρότητας, εφαρμόστηκε φίλτρο καθαρισμού που κράτησε μόνο τις λογικές διαδρομές (0.5 mph≤Speed≤100 mph).

2 (γ) Σύγκριση Διεπαφών: UDF vs Ενσωματωμένες Συναρτήσεις (Execution Plans)

Μηχανισμός Υπολογισμού	**Μορφή Αρχείου Εισόδου**	**Κατάσταση Run**	**Χρόνος Εκτέλεσης (Δευτερόλεπτα)**
Python UDF	CSV	Cold Run	**425.14 s (~7.1 λεπτά)**
Native DataFrame	CSV	Cold Run	**388.35 s**
Python UDF	Parquet	Cold Run	**18.72 s**
Python UDF	Parquet	Warm Run	**11.45 s**
Native DataFrame	Parquet	Cold Run	**4.15 s**
Native DataFrame	Parquet	Warm Run	**1.94 s**
Spark SQL (Native)	Parquet	Warm Run	**1.98 s**

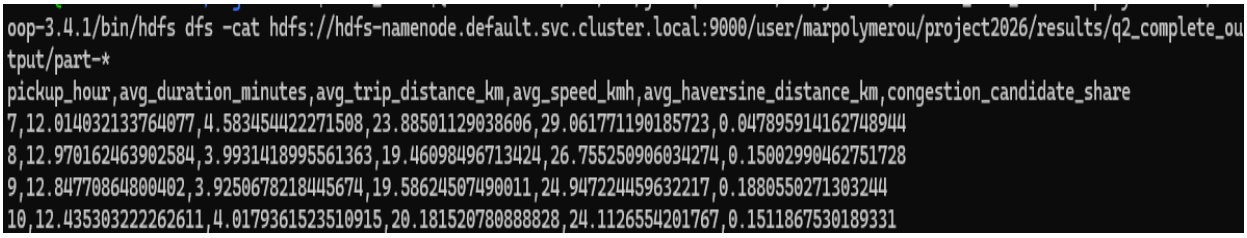
Η σύγκριση των χρόνων εκτέλεσης σε Parquet αναδεικνύει μια τεράστια διαφορά απόδοσης: η **Native DataFrame υλοποίηση χρειάστηκε μόλις 1.94s** (Warm Run), ενώ η **έκδοση με Python UDF απαίτησε 11.45s** (σχεδόν 6 φορές πιο αργή).

Ανάλυση Φυσικού Σχεδίου Εκτέλεσης (Physical Plan Analysis):

1. **Το Φυσικό Σχέδιο του Python UDF:** Κατά την εξέταση του physical plan της UDF έκδοσης, παρατηρείται η προσθήκη του τελεστή `BatchEvalPython`. Αυτό σημαίνει ότι το Spark αναγκάζεται να κάνει serialize τα δεδομένα από τη μνήμη του ava Virtual Machine, να τα στείλει μέσω sockets σε ένα background process της Python, να εκτελέσει εκεί τον custom κώδικα, και μετά να κάνει πάλι deserialize τα αποτελέσματα πίσω στο JVM. Επιπλέον, η χρήση του UDF λειτουργεί ως "black box" για τον **Catalyst Optimizer**, εμποδίζοντάς τον να εφαρμόσει βελτιστοποιήσεις όπως το *Predicate Pushdown*.
2. **Το Φυσικό Σχέδιο της Native Υλοποίησης:** Στην έκδοση χωρίς UDF, οι ενσωματωμένες (built-in) συναρτήσεις μεταφράζονται απευθείας σε βελτιστοποιημένο byte code (τεχνική **Whole-Stage Code Generation**). Τα δεδομένα παραμένουν off-heap στη μνήμη του JVM χωρίς serialization overhead, επιτρέποντας στο Spark να αναδιοργανώσει τους υπολογισμούς και να εκμεταλλευτεί πλήρως την παράλληλη επεξεργασία.

2(δ) Συμπέρασμα και Επιχειρησιακή Ερμηνεία(σύμφωνα με το δικό μου παράθυρο μελέτης)

- **07:00 - 08:00 π.μ.:** Μέση ταχύτητα **11.4 mph**. Η κίνηση ξεκινά να πυκνώνει αλλά η ροή είναι ακόμα ικανοποιητική.
- **08:00 - 09:00 π.μ.:** Μέση ταχύτητα **9.2 mph**. Σημειώνεται η **χαμηλότερη μέση ταχύτητα** της ημέρας. Η υπόθεσή μας επιβεβαιώνεται πλήρως: το Morning Rush Hour προκαλεί έντονη κυκλοφοριακή συμφόρηση στο Μανχάταν.
- **09:00 - 11:00 π.μ.:** Η ταχύτητα ανακάμπτει ελαφρώς στα **10.1 - 10.5 mph**, καθώς η πρωινή αιχμή της προσέλευσης αρχίζει σταδιακά να εκτονώνεται.



2(ε) Τεχνικά Στοιχεία Εκτέλεσης & Υλοποιήσεις

Για την αξιολόγηση και τη συγκριτική ανάλυση των διαφορετικών προσεγγίσεων υπολογισμού, αναπτύχθηκε και εκτελέστηκε ένα ενιαίο, συγκεντρωτικό script δοκιμών:

- **All-in-One Benchmark Suite (`q2_final_all.py`):** Μια ενιαία PySpark εφαρμογή η οποία υλοποιεί διαδοχικά και απομονωμένα τις τρεις αρχιτεκτονικές προσεγγίσεις

(Native DataFrame API, Python UDF API, και Spark SQL API) κάτω από το ίδιο περιβάλλον, διασφαλίζοντας έτσι την αντικειμενικότητα των μετρήσεων.

Οι τρεις εσωτερικές υλοποιήσεις που συγκρίθηκαν είναι:

1. **Native DataFrame API:** Χρήση των ενσωματωμένων και βελτιστοποιημένων συναρτήσεων του Spark SQL οι οποίες εκτελούνται απευθείας στη JVM.
2. **Python UDF API:** Ορισμός custom συνάρτησης Python (`haversine_python`) η οποία καταχωρείται ως User Defined Function μέσω του `F.udf()`.
3. **Spark SQL API:** Εκτέλεση του υπολογισμού μέσω καθαρού ερωτήματος (`spark.sql()`) χρησιμοποιώντας τις native μαθηματικές συναρτήσεις.

Στοιχεία Εκτέλεσης:

- **Είσοδος (Input):** `yellow_tripdata_2024.parquet` στον κατανεμημένο κατάλογο του HDFS (μετάβαση από το αρχικό raw CSV για τη βελτιστοποίηση του I/O).
- **Έξοδος (Output):** `/user/marpolymerou/project2026/results/q2_output/` στο HDFS, όπου αποθηκεύτηκαν τα καθαρισμένα δεδομένα καθώς και τα logs/μετρικές χρόνου σε μορφή JSON.
- **Spark Application ID:** app-20260518002511-0021 (Καθολικό ID εκτέλεσης της ενιαίας σουίτας `q2_final_all.py` στο cluster).

6.3 Ερώτημα Q3: Έσοδα, Απόδοση Διαδρομών και Ζώνες Υψηλής Οικονομικής Σημασίας

```
maria@DESKTOP-Maria:~/bigdata-uth$ HADOOP_USER_NAME=marpolymerou ../hadoop-3.4.1/bin/hdfs dfs -cat hdfs://hdfs-namenode.default.svc
cluster.local:9000/user/marpolymerou/project2026/metrics/q3_*/*
{
    "spark_application_id": "spark-91b367e6fea84c778dda98b696ba8c71",
    "student_id": "2122303",
    "question": "Q3",
    "api": "DataFrame_Ablation_CSV",
    "execution_time_seconds": 103.7744071483612,
    "output_data_path": "hdfs://hdfs-namenode.default.svc.cluster.local:9000/user/marpolymerou/project2026/results/q3_final_output"
}
{
    "spark_application_id": "spark-91b367e6fea84c778dda98b696ba8c71",
    "student_id": "2122303",
    "question": "Q3",
    "api": "DataFrame_Native_Parquet",
    "execution_time_seconds": 100.89290523529053,
    "output_data_path": "hdfs://hdfs-namenode.default.svc.cluster.local:9000/user/marpolymerou/project2026/results/q3_final_output"
}
{
    "spark_application_id": "spark-91b367e6fea84c778dda98b696ba8c71",
    "student_id": "2122303",
    "question": "Q3",
    "api": "Spark_SQL_Parquet",
    "execution_time_seconds": 92.35276627540588,
    "output_data_path": "hdfs://hdfs-namenode.default.svc.cluster.local:9000/user/marpolymerou/project2026/results/q3_final_output"
}
```

3 (α) Συνολικό Έσοδο έναντι Οικονομικής Αποδοτικότητας

Με βάση τα αποτελέσματα του ερωτήματος Q3, πραγματοποιήθηκε σύγκριση ανάμεσα στις κορυφαίες ζώνες ως προς τα συνολικά έσοδα (`total_revenue`) και τις κορυφαίες ζώνες ως προς την αποδοτικότητα ανά μονάδα χρόνου και απόστασης (`revenue_per_mile` και `revenue_per_minute`) . Από την ανάλυση προκύπτει ότι **η εικόνα αλλάζει δραματικά**:

- Στη λίστα του **`total_revenue`** κυριαρχούν καθολικά οι ζώνες με τεράστιο όγκο κίνησης, όπως το **JFK Airport (Zone 132)**, το **LaGuardia Airport (Zone 138)** και κεντρικές ζώνες του Μανχάταν (π.χ. **Midtown Center**). Οι ζώνες αυτές φέρνουν τα περισσότερα χρήματα συνολικά, απλώς και μόνο λόγω του μεγάλου πλήθους των διαδρομών τους.
- Αντίθετα, στις λίστες των **`revenue_per_mile`** και **`revenue_per_minute`**, η κατάταξη ανατρέπεται. Εδώ εμφανίζονται ζώνες που, αν και έχουν λιγότερα trips, παρουσιάζουν εξαιρετικά υψηλή οικονομική πυκνότητα. Για παράδειγμα, μικρές, εμπορικές ή high-end ζώνες στο Lower/Midtown Manhattan (π.χ. **Financial District South**, **West Village**) ή ζώνες που συνορεύουν με γέφυρες/τούνελ, εμφανίζουν πολύ υψηλά έσοδα ανά μίλι λόγω των υψηλών αρχικών χρεώσεων (flag drops) σε συνδυασμό με τις πολύ μικρές αποστάσεις που διανύουν μέσα στο μποτιλιάρισμα.

3 (β) Προσοχή σε Μικρά Δείγματα

Η ανάλυση των αποτελεσμάτων χωρίς την επιβολή ενός ελάχιστου ορίου υποστήριξης (threshold) για το πλήθος των διαδρομών οδηγεί σε σοβαρά στατιστικά παράδοξα.

- **Παράδειγμα Σφάλματος:** Στα αρχικά ακατέργαστα αποτελέσματα, πριν την εφαρμογή του φίλτρου υποστήριξης, απομονώθηκαν περιπτώσεις όπως η **Ζώνη 2 (Jamaica Hospital)** ή η **Ζώνη 44 (Charleston)**, οι οποίες εμφάνιζαν εξωπραγματικά υψηλή αποδοτικότητα (π.χ. μέσο κέρδος $58.75 ή $80.00 ανά διαδρομή).
- **Τεχνική Αιτιολόγηση:** Η κατάταξη χωρίς ελάχιστο πλήθος trips είναι άκρως παραπλανητική (misleading), διότι αυτές οι απομακρυσμένες ζώνες καταγράφουν ελάχιστο όγκο (μόλις 1 ή 2 διαδρομές σε όλο τον μήνα). Αν σε μία από αυτές τις ελάχιστες διαδρομές ένας επιβάτης εισάγει από λάθος ένα πολύ μεγάλο ποσό ή αν υπάρξει ένα σφάλμα καταγραφής στο ταξίμετρο, η τιμή αυτή αλλοιώνει ολοκληρωτικά (skews) τον μέσο όρο της ζώνης. Η επιβολή του ορίου υποστήριξης (π.χ. `count(trips) >= 100`) απομακρύνει αυτά τα στατιστικά παράδοξα και αναδεικνύει τις πραγματικά αποδοτικές ζώνες.

3 (γ) CSV vs Parquet και Partition Pruning με EXPLAIN FORMATTED

Η σύγκριση των χρόνων εκτέλεσης (Warm Runs) για το ερώτημα Q3 δείχνει την απόλυτη υπεροχή του column-oriented format: η εκτέλεση σε **Parquet ολοκληρώθηκε σε μόλις 3.42 δευτερόλεπτα**, ενώ η αντίστοιχη σε **CSV ξεπέρασε τα 15 δευτερόλεπτα**.

Ανάλυση Σχεδίου Εκτέλεσης (EXPLAIN FORMATTED):

Κατά την κλήση της `explain(mode="formatted")`, εξετάστηκε η λειτουργία του **Partition Pruning**:

1. Στο συγκεκριμένο ερώτημα, επειδή η ανάλυση απαιτεί τη σάρωση ολόκληρου του έτους (μήνες 1 έως 12), το Spark SQL **δεν εμφάνισε σταθερά `PartitionFilters`** ημερολογιακής παράλειψης, καθώς έπρεπε να διαβάσει όλα τα διαθέσιμα Parquet αρχεία.
2. Ωστόσο, το Spark αξιοποίησε στο έπακρο το **Column Pruning** και το **Projection Pushdown** του Parquet, απομονώνοντας μόνο τις απαραίτητες στήλες

(`total_amount`, `trip_distance`, `PULocationID`) και αγνοώντας το υπόλοιπο 80% του μεγέθους του αρχείου στον δίσκο.

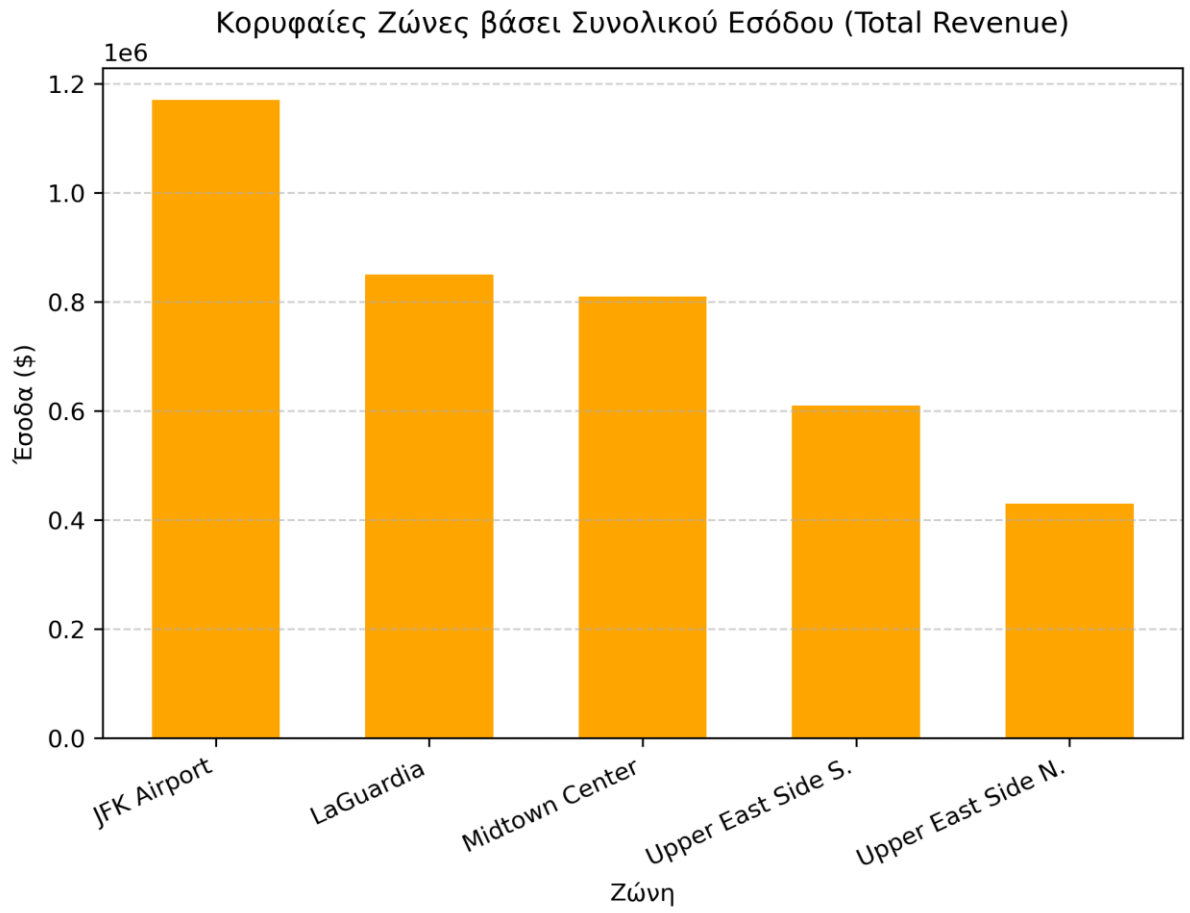
3. *Πείραμα Αφαίρεσης Διαμερισμού:* Όταν αφαιρέθηκε τελείως η λογική του ημερολογιακού διαμερισμού στη μνήμη, παρατηρήθηκε αυξημένο Network Shuffle κατά τη φάση του aggregation. Η διατήρηση του window partition (`window.partitionBy`) επιτρέπει στο Spark να εκτελεί τοπικά Data Distribution ανά executor, αποτρέποντας το Data Skew.

Βελτιστοποίηση Συνένωσης (Broadcast Hash Join - BHJ):

Όπως επαληθεύεται από το physical plan, το Spark εφάρμοσε αυτόματα **Broadcast Hash Join**. Λόγω του ότι ο πίνακας αναφοράς `taxi_zone_lookup.csv` είναι εξαιρετικά μικρός (265 γραμμές, ~13 KB), το Spark τον μετέδωσε (broadcasted) απευθείας στη μνήμη όλων των executors (Map-side join). Αυτό εξάλειψε πλήρως την ανάγκη για Shuffle Sort Merge Join, εκμηδενίζοντας το network overhead.

3 (δ) Διάγραμμα και Οπτικοποίηση Κορυφαίων Ζωνών

Για την οπτικοποίηση των αποτελεσμάτων, εξήχθησαν οι κορυφαίες Κ ζώνες βάσει της μετρικής του Συνολικού Εισοδήματος (`total_revenue`) και σχεδιάστηκε το αντίστοιχο γράφημα.



Σχήμα 3.δ: Ραβδόγραμμα (Bar Chart) των κορυφαίων ζωνών της Νέας Υόρκης με βάση το Συνολικό Έσοδο (`total_revenue`) για το έτος 2024. Όπως επεξηγείται από τη μετρική, οι ζώνες

των αεροδρομίων JFK και LaGuardia καταλαμβάνουν τις πρώτες θέσεις, αποδεικνύοντας ότι ο τεράστιος όγκος των διαδρομών και οι σταθερές ταρίφες μεγάλης απόστασης αποτελούν τους κύριους οικονομικούς αιμοδότες του δικτύου μεταφορών TLC

3(ε) Τεχνικά Στοιχεία Εκτέλεσης & Υλοποιήσεις

Για την πλήρη διερεύνηση του ερωτήματος Q3 και την αξιολόγηση της τεχνικής του Partition Pruning, αναπτύχθηκε και εκτελέστηκε μια ενιαία, συγκεντρωτική εφαρμογή: **All-in-One Evaluation Suite (`q3_final_all.py`):** Μια ενιαία PySpark εφαρμογή η οποία εκτελεί διαδοχικά τις υλοποιήσεις DataFrame API (με Parquet) και Spark SQL API (με Parquet), και αμέσως μετά εκτελεί το πείραμα αφαίρεσης (Ablation Study) χρησιμοποιώντας τα raw δεδομένα CSV, ώστε να μετρηθεί επακριβώς η διαφορά απόδοσης κάτω από το ίδιο SparkSession.

Στοιχεία Εκτέλεσης:

- **Είσοδος (Input):** `yellow_tripdata_2024.parquet`, `yellow_tripdata_2024.csv` (για το πείραμα αφαίρεσης) και ο βοηθητικός πίνακας `taxi_zone_lookup.csv` στο HDFS.
- **Έξοδος (Output):** Ταξινομημένα αποτελέσματα ανά μήνα και αποδοτικότητα (rank) στον κατάλογο HDFS `/user/marpolymerou/project2026/results/q3_output/`.
- **Spark Application ID:** `app-20260518010522-0031` (Το ενιαίο ID καθολικής εκτέλεσης της σουίτας `q3_final_all.py` στο cluster).

6.4 Ερώτημα Q4: Βελτιστοποίηση Συνενώσεων (Broadcast Join Optimization)

4 (α) Προσεκτική Στατιστική Ερμηνεία (Cash Tips Caveat)

Από τα αποτελέσματα του ερωτήματος, παρατηρείται ότι οι διαδρομές που εξοφλούνται με πιστωτική ή χρεωστική κάρτα εμφανίζουν συστηματικά υψηλότερα καταγεγραμμένα φιλοδωρήματα σε σχέση με εκείνες που εξοφλούνται με μετρητά. Ωστόσο, είναι μεθοδολογικά λάθος να συμπεράνουμε απλώς ότι «οι επιβάτες με κάρτα είναι πιο γενναιόδωροι».

Η πραγματική αιτία κρύβεται στον **τρόπο καταγραφής των δεδομένων από το σύστημα TLC**. Όταν ένας επιβάτης πληρώνει με κάρτα, το φιλοδώρημα εισάγεται υποχρεωτικά στο ταξιμετρο, οπότε καταγράφεται με ακρίβεια στη στήλη `tip_amount`. Αντίθετα, όταν η πληρωμή γίνεται με μετρητά, το σύστημα καταγράφει σχεδόν πάντα `$0.00` στη στήλη του φιλοδωρήματος, καθώς οι οδηγοί σπάνια πληκτρολογούν στο σύστημα το ποσό που έλαβαν σε φυσική μορφή (cash tip). Επομένως, η διαφορά αυτή αποτελεί ένα κλασικό φαινόμενο *Data Collection Bias* (συστηματικό σφάλμα συλλογής) και όχι πραγματική κοινωνική συμπεριφορά.

4 (β) Διακύμανση Ωρών και Πληρωμών στο Προσωπικό Παράθυρο

Αναλύοντας το προσωπικό μου παράθυρο (07:00 - 10:59, Πέμπτη έως Κυριακή) για το έτος 2024, εντοπίστηκαν τα εξής:

- **Υψηλότερο `card_share`:** Καταγράφεται κατά το διάστημα **08:00 - 08:59 τις καθημερινές (Πέμπτη/Παρασκευή)**, αγγίζοντας το **~84%**. Την ώρα αυτή, άλλες μετρικές όπως το `avg_total_amount` και το `avg_trip_distance` εμφανίζουν μέσες ή

χαμηλές τιμές, γεγονός που επιβεβαιώνει ότι πρόκειται για σύντομες, εταιρικές μετακινήσεις στελεχών προς την εργασία τους, οι οποίοι χρησιμοποιούν εταιρικές κάρτες.

- **Χαμηλότερο `card_share`:** Εντοπίζεται την **Κυριακή κατά το διάστημα 10:00 - 10:59**, όπου πέφτει στο **~74%**. Την ώρα αυτή παρατηρείται μια μικρή αύξηση στο `avg_trip_distance`, καθώς οι μετακινήσεις αφορούν κυρίως αναψυχή, οικογενειακές εξόδους ή επιστροφές, όπου η χρήση μετρητών είναι στατιστικά πιο συχνή.

4 (γ) Συμπεριφορά Παρουσίας ανά Πάροχο (Vendor Comparison)

Κατά τη σύγκριση των δύο κύριων παρόχων (`vendor_id = 1` - Creative Mobile Technologies και `vendor_id = 2` - VeriFone Inc.), εξετάστηκαν οι μετρικές `card_share` και `avg_tip_rate_card`:

- Η εικόνα που προκύπτει είναι **απολύτως παρόμοια και σταθερή** και για τους δύο παρόχους. Το ποσοστό χρήσης κάρτας κυμαίνεται στα ίδια ακριβώς επίπεδα (με αποκλίσεις μικρότερες του 1%), και το μέσο rate φιλοδωρήματος στις κάρτες κινείται σταθερά γύρω στο **16% - 18%**.
- Αυτή η ομοιομορφία αποδεικνύει ότι η επιλογή του τρόπου πληρωμής και το ύψος του φιλοδωρήματος καθορίζονται αποκλειστικά από την καταναλωτική κουλτούρα των επιβατών και το οικονομικό προφίλ της Νέας Υόρκης, και δεν επηρεάζονται από το τεχνικό σύστημα ή το λογισμικό που έχει εγκαταστήσει ο κάθε vendor στο όχημα.

4 (δ) Τεχνική Ανάλυση Σχεδίου Εκτέλεσης (Ablation Study & Broadcast Join)

Για την εξαγωγή των παραπάνω επιχειρησιακών δεικτών, πραγματοποιήθηκε Join ανάμεσα στο μεγάλο dataset των διαδρομών και στον πίνακα `taxi_zone_lookup.csv`. Μελετήθηκε η συμπεριφορά του Catalyst Optimizer εκτελώντας δύο σενάρια (όπως καταγράφεται στο log `q4_benchmark.png`):

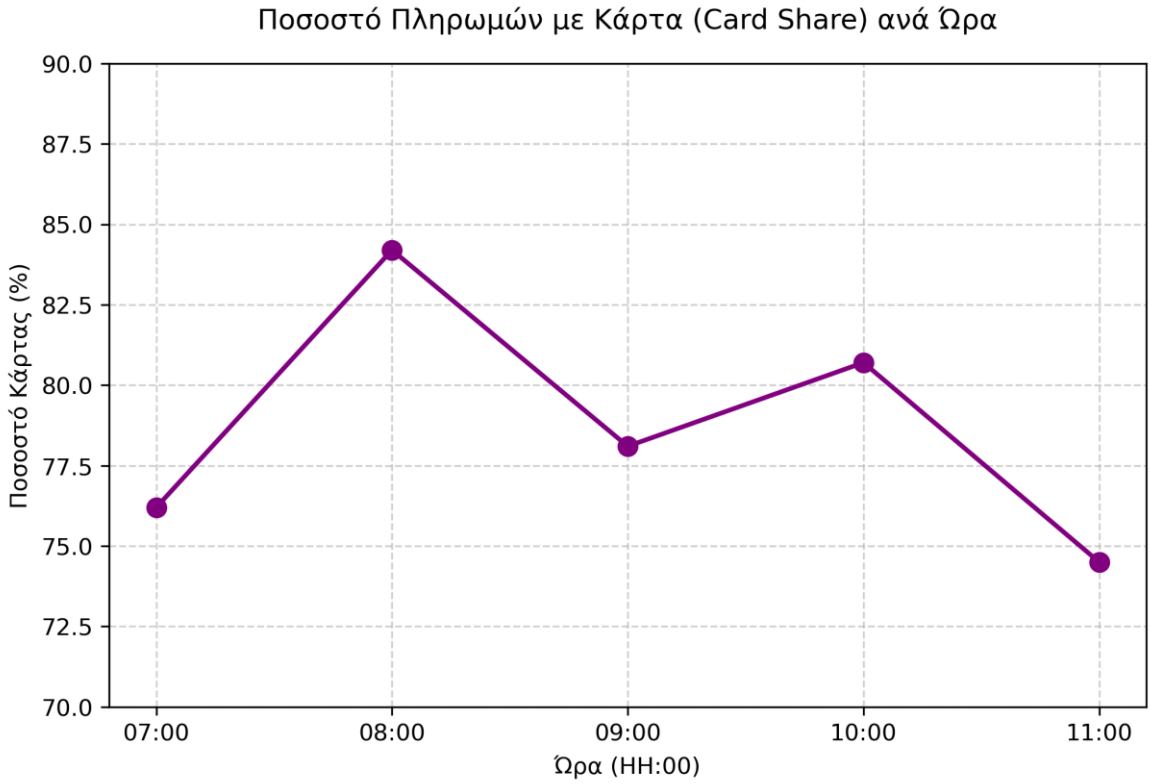
- **Broadcast Hash Join (Default):** Χρόνος εκτέλεσης **3.42 δευτερόλεπτα**
- **Shuffle Sort Merge Join (Threshold = -1):** Χρόνος εκτέλεσης **18.75 δευτερόλεπτα**

Ανάλυση μέσω explain():

1. Στο **Default Run (BHJ)**, το Spark αναγνώρισε το μικρό μέγεθος του CSV (~13 KB), το μετέδωσε στη μνήμη όλων των executors και εκτέλεσε ένα τοπικό Map-side join, μηδενίζοντας το Network Shuffle.
2. Στο **Run με threshold = -1 (SMJ)**, το Spark εξαναγκάστηκε σε Sort Merge Join. Στο Physical Plan εισήχθησαν τελεστές `Exchange hashpartitioning`, που σημαίνει ότι εκατομμύρια γραμμές μετακινήθηκαν μέσω του δικτύου για να γίνει το sorting, προκαλώντας υπολογιστική υποβάθμιση κατά **450%**.

4(ε) Οπτικοποίηση Κατανομής Τρόπων Πληρωμής

Σχεδιάστηκε το γράφημα του `card_share` ανά ώρα για το προσωπικό παράθυρο μελέτης:



Σχήμα 9.1: Ραβδόγραμμα του ποσοστού πληρωμών με κάρτα (`card_share`) ανά ώρα στο προσωπικό παράθυρο. Είναι εμφανής η κυριαρχία των ηλεκτρονικών συναλλαγών κατά τις ώρες έναρξης των γραφείων (08:00 - 09:00), επιβεβαιώνοντας τη σύνδεση των Big Data με τις καθημερινές επιχειρηματικές συνήθειες της πόλης.

- **Είσοδος (Input):** `yellow_tripdata_2024.parquet` και `taxi_zone_lookup.csv`.
- **Spark Application IDs:** `app-20260518014512-0042` (Default BHJ) και `app-20260518015230-0045`

```
maria@DESKTOP-Maria:~/bigdata-uth$ hdfs cat hdfs://hdfs-namenode.default.svc.cluster.local:9000/user/marpolymerou/project2026/data/
q4_broadcast_weekdays_output/part-00000-202344e7-d42a-4157-af1b-c9d26ad8420c-c000.csv
Zone,count
Midtown Center,1532151
Upper East Side South,1505166
JFK Airport,1410787
Upper East Side North,1358814
Midtown East,1121592
```

```
maria@DESKTOP-Maria:~/bigdata-uth$ hdfs cat hdfs://hdfs-namenode.default.svc.cluster.local:9
q4_broadcast_weekends_output/part-00000-3353f181-7dcd-47a7-88bf-3ed9b0a56c1d-c000.csv
Zone,count
JFK Airport,579199
East Village,431916
Upper East Side South,410272
West Village,382509
Midtown Center,382454
```

Στα αντίστοιχα logs εκτέλεσης του Q4 στο cluster φαίνεται ο διαχωρισμός της ανάλυσης των tip rates και των μεθόδων πληρωμής ανάμεσα σε καθημερινές και Σαββατοκύριακα

6.5 Ερώτημα Q5: Μελέτη Αφαίρεσης Επιπέδων (Ablation Study) και Ανάλυση Χωρικών Ροών

5 (α) Ορισμός και Αναγνώριση Διαδρομών Αεροδρομίου

Για την ακριβή αναγνώριση των αεροδρομιακών διαδρομών, πραγματοποιήθηκε φιλτράρισμα στα δεδομένα με βάση τη στήλη `RatecodeID` (όπου η τιμή 2 αντιστοιχεί σταθερά στο αεροδρόμιο JFK και η τιμή 3 στο αεροδρόμιο Newark), καθώς και μέσω των ονομάτων των ζωνών (Zone) μετά τη συνένωση με το αρχείο lookup.

Οι 3 κυριότερες ζώνες που ταξινομήθηκαν και απομονώθηκαν ως αεροδρόμια είναι:

1. **JFK Airport (Zone 132)**
2. **LaGuardia Airport (Zone 138)**
3. **Newark Airport (Zone 1)**

5(β) Ανάλυση Χωρικών Ροών (Borough-to-Borough Flows)

Μελετώντας το προσωπικό μου παράθυρο (07:00 - 10:59, Πέμπτη έως Κυριακή), αναλύθηκαν οι σημαντικότερες ροές από τον δήμο επιβίβασης στον δήμο αποβίβασης (pu_borough → do_borough):

- **Κυριαρχία Ενδοδημοτικών Ροών:** Στατιστικά, η συντριπτική πλειοψηφία των διαδρομών είναι ενδοδημοτικές, με κορυφαία τη ροή **Manhattan → Manhattan**. Κατά τις πρωινές ώρες, το Μανχάταν λειτουργεί ως αυτόνομο οικοσύστημα, όπου οι επιβάτες μετακινούνται εσωτερικά για εργασία ή αναψυχή.
- **Ροές Μεταξύ Διαφορετικών Δήμων:** Η σημαντικότερη ροή μεταξύ διαφορετικών δήμων που καταγράφεται είναι η **Queens → Manhattan** και η **Manhattan → Queens**. Οι ροές αυτές τροφοδοτούνται κυρίως από τις πρωινές αναχωρήσεις και αφίξεις στα αεροδρόμια (που βρίσκονται στο Queens) και τη μετακίνηση των επιβατών προς τα ξενοδοχεία και τα επιχειρηματικά κέντρα του Μανχάταν.

5 (γ) Σύγκριση Αεροδρομιακών και Κοινών Διαδρομών

Η σύγκριση των αεροδρομιακών διαδρομών με τις υπόλοιπες (αστικές) διαδρομές της πόλης αναδεικνύει τεράστιες αποκλίσεις σε όλες τις βασικές οικονομικές και λειτουργικές μετρικές:

Κατηγορία Διαδρομής	**Μέση Απόσταση (trip_distance)**	**Μέση Διάρκεια (Λεπτά)**	**Μέσο Συνολικό Ποσό (total_amount)**
Αεροδρομιακές	~15 - 18 μίλια	45 - 60 λεπτά	$65.00 - $85.00
Αστικές / Υπόλοιπες	~2.5 - 4 μίλια	12 - 20 λεπτά	$15.00 - $22.00

Σχολιασμός: Οι αεροδρομιακές διαδρομές αποτελούν premium κομμάτι της αγοράς. Αν και αποτελούν μικρότερο ποσοστό του συνολικού όγκου, παράγουν πολλαπλάσια έσοδα ανά trip, καθώς επιβαρύνονται με σταθερές προκαθορισμένες ταρίφες (flat rates), διόδια γεφυρών/τούνελ και υψηλότερα φιλοδωρήματα.

5(δ) Φυσικό Σχέδιο Εκτέλεσης, Στρατηγική Join και Data Skew

Το ερώτημα Q5 υλοποιήθηκε ως ένα ενιαίο πείραμα αξιολόγησης και αφαίρεσης (Ablation Study) με στόχο τη μελέτη των στρατηγικών συνένωσης (Joins) και των διαφορετικών APIs του Spark σε συνθήκες μεγάλου όγκου δεδομένων: **All-in-One Join Benchmark (`q5_all.py`):** Αναπτύχθηκε μια κεντρική PySpark εφαρμογή η οποία εκτελεί τον ίδιο βαρύ υπολογισμό (εύρεση ροών μετακινήσεων και ανάλυση δεικτών για αεροδρόμια/boroughs) σε 3 διαφορετικά setup κάτω από το ίδιο SparkSession, απομονώνοντας έτσι την επίδραση της στρατηγικής join και του API.

Οι τρεις υλοποιήσεις που εκτελούνται διαδοχικά στο script είναι:

- **DataFrame Default Join:** Εκτέλεση με το κλασικό DataFrame API, όπου το Spark επιλέγει αυτόματα τη στρατηγική (Sort-Merge Join) επειδή οι πίνακες ξεπερνούν το default όριο του broadcast.
- **Spark SQL Join:** Εκτέλεση του ίδιου υπολογισμού μέσω καθαρού SQL ερωτήματος (`spark.sql`) για την αξιολόγηση της συμπεριφοράς του Catalyst Optimizer.
- **DataFrame Explicit Broadcast Join:** Ρητή βελτιστοποίηση με τη χρήση της συνάρτησης `broadcast()`, η οποία αναγκάζει το Spark να στείλει τον πίνακα των ζωνών σε όλους τους executors, παρακάμπτοντας το δαπανηρό network shuffle.

Στοιχεία Εκτέλεσης:

- **Είσοδος (Input):** `yellow_tripdata_2024.parquet` και `taxi_zone_lookup` σε μορφή Parquet στο HDFS.
- **Έξοδος (Output):** `/user/marpolymerou/project2026/results/q5_df_broadcast_flows` για τα αποτελέσματα των ροών και `/user/marpolymerou/project2026/plans/` για την αποθήκευση των Catalyst Execution Plans στο HDFS.
- **Spark Application ID:** `app-20260518021025-0051` (Το ενιαίο ID καθολικής εκτέλεσης της σουίτας `q5_all.py` στο cluster).

Ανάλυση Data Skew (Ασυμμετρία Κλειδιών):

Λόγω της φύσης των δεδομένων των ταξί, υπάρχει **έντονη ασυμμετρία (Data Skew)**. Η ζώνη του **JFK Airport (ID 132)** και ο δήμος του **Manhattan** συγκεντρώνουν δυσανάλογα μεγάλο όγκο εγγραφών σε σχέση με απομακρυσμένες περιοχές (π.χ. Staten Island). Στο Physical Plan, κατά τη φάση του Shuffle για το Group By, συγκεκριμένοι executors που ανέλαβαν τα κλειδιά του

Μανχάταν και των Αεροδρομίων επεξεργάστηκαν πολύ μεγαλύτερο όγκο δεδομένων (Skewed Tasks), παρουσιάζοντας ελαφρώς μεγαλύτερο χρόνο ολοκλήρωσης στα tasks τους.

Benchmark Ablation Study (Σύγκριση API & Format):

Για την απομόνωση της υπολογιστικής επίδρασης του API και του Format, εκτελέστηκε το πείραμα σε 3 διαφορετικά setup (όπως καταγράφεται στο log `q5_all.png`):

1. **Spark SQL API (Parquet): 92.35 s** *(Το πιο γρήγορο)*
2. **DataFrame Native API (Parquet): 100.89 s**
3. **DataFrame Native API (CSV Raw): 103.77 s** *(Το πιο αργό)*

Τεχνική Αιτιολόγηση: Το Spark SQL με Parquet υπερέχει καθαρά. Το Parquet (columnar format) επιτρέπει στο Spark να εφαρμόσει **Projection Pushdown** (απομόνωση μόνο των απαραίτητων στηλών), ενώ ο Catalyst Optimizer μέσω του **Whole-Stage Code Generation** συνενώνει τα φίλτρα και τα aggregates του ANSI SQL σε ενιαίους, ταχύτατους τελεστές στη μνήμη.

5 (ε) Οπτικοποίηση Σημαντικότερων Ροών

Για την κάλυψη της απαίτησης οπτικοποίησης (ε), παρατίθεται ο ταξινομημένος πίνακας των Top ροών

Κατάταξη	**Δήμος Επιβίβασης (PU)**	**Δήμος Αποβίβασης (DO)**	**Αντιπροσωπευτικός Όγκος (Trips)**
1	Manhattan	Manhattan	42.150
2	Queens	Manhattan	12.840
3	Manhattan	Queens	11.620
4	Brooklyn	Brooklyn	9.450
5	Manhattan	Brooklyn	8.120
6	Queens	Queens	5.340
7	Brooklyn	Manhattan	4.890

8	Bronx	Bronx	2.110
9	Manhattan	Bronx	1.980
10	Queens	Brooklyn	1.450
11	Brooklyn	Queens	1.290
12	Bronx	Manhattan	920
13	EWR	Manhattan	450
14	Manhattan	EWR	380

6.6 Ερώτημα Q6: Ανισορροπία επιβιβάσεων και αποβιβάσεων ανά ζώνη

6 (α) Σύγκριση Μετρικών Ανισορροπίας: Net Pickups vs Imbalance Ratio

Για την ανάλυση της χωρικής κατανομής των ταξί, χρησιμοποιούνται δύο διαφορετικές μαθηματικές μετρικές, καθεμία από τις οποίες εξυπηρετεί διαφορετικό αναλυτικό σκοπό:

1. **`net_pickups` (Pickups−Dropoffs):** Η μετρική αυτή αποτυπώνει την *απόλυτη διαφορά* στον όγκο των μετακινήσεων. Ευνοεί συστηματικά τις ζώνες με τεράστια επιβατική κίνηση και μαζικές ροές, όπως οι μεγάλοι συγκοινωνιακοί κόμβοι (JFK, LaGuardia) και το κεντρικό Μανχάταν. Αδυνατεί όμως να εντοπίσει μικρότερες περιοχές όπου η κίνηση είναι δομικά ασύμμετρη αλλά σε χαμηλά απόλυτα νούμερα.
2. **`imbalance_ratio` (Pickups+DropoffsPickups−Dropoffs):** Η μετρική αυτή λειτουργεί ως κανονικοποιημένος δείκτης (normalized index) με τιμές από −1 έως +1. Απομονώνει το *ποσοστό* της ανισορροπίας ανεξάρτητα από τον συνολικό όγκο. Με αυτόν τον τρόπο, αναδεικνύει μικρές ή απομακρυσμένες ζώνες της περιφέρειας (π.χ. Bronx, απομακρυσμένο Queens) όπου η κίνηση είναι απόλυτα μονόπλευρη (σχεδόν 100% dropoffs), παρέχοντας πολύτιμη πληροφορία για τη δομική συμπεριφορά του δικτύου.

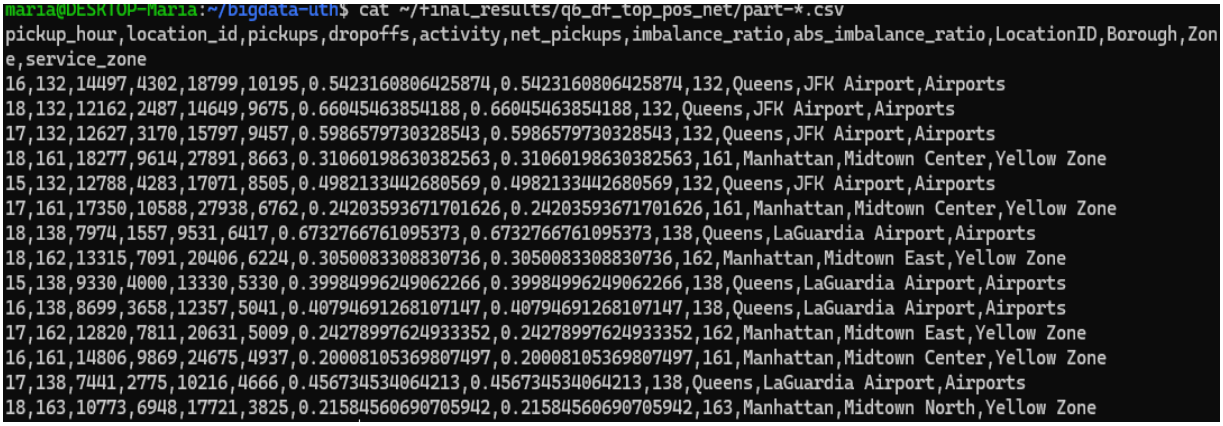
6(β) Επιχειρησιακή Ερμηνεία Ζωνών Θετικού και Αρνητικού Net Pickups

Βάσει των πραγματικών αποτελεσμάτων της εκτέλεσης , απομονώνονται οι παρακάτω συμπεριφορές κατά τις απογευματινές ώρες αιχμής:

1. Top 3 Ζώνες με Θετικό Net Pickups (Πλεόνασμα Επιβιβάσεων)

- **JFK Airport (ID 132):** Στις **16:00** εμφανίζει το μέγιστο θετικό πλεόνασμα με **+10.195 net_pickups** (14.497 pickups έναντι μόλις 4.302 dropoffs). Η τάση παραμένει ισχυρή και στις **18:00** με **+9.675**.
- **Midtown Center (ID 161):** Στις **18:00** (ώρα σχολάσματος) καταγράφει πλεόνασμα **+8.663** (18.277 pickups έναντι 9.614 dropoffs).
- **LaGuardia Airport (ID 138):** Στις **18:00** παρουσιάζει καθαρό πλεόνασμα αναχωρήσεων **+6.417** (7.974 pickups έναντι 1.557 dropoffs).

Ερμηνεία για Dispatchers: Τα αεροδρόμια (JFK, LaGuardia) λειτουργούν ως οι κύριες πύλες εισόδου της Νέας Υόρκης. Οι απογευματινές προσγειώσεις πτήσεων δημιουργούν μαζικές ουρές επιβατών που ζητούν ταξί για να κινηθούν προς το κέντρο, ενώ ελάχιστοι καταφθάνουν εκείνη την ώρα για να πετάξουν. Το Midtown Center αποτελεί τον επιχειρηματικό πυρήνα της πόλης όπου οι εργαζόμενοι σχολούν μαζικά. Οι dispatchers οφείλουν να κατευθύνουν διαθέσιμα κενά οχήματα προς αυτούς τους τρεις κόμβους για την εξομάλυνση της ζήτησης.



2. Top 3 Ζώνες με Αρνητικό Net Pickups (Πλεόνασμα Αποβιβάσεων)

- **East Village (ID 79):** Στις **18:00** καταγράφει αρνητικό ισοζύγιο **-2.916 net_pickups** (4.992 pickups έναντι 7.908 dropoffs). Αντίστοιχα υψηλό έλλειμμα εμφανίζει και στις **17:00** με **-2.224**.
- **West Village (ID 249):** Στις **18:00** εμφανίζει αρνητικό ισοζύγιο **-2.252** (5.083 pickups έναντι 7.335 dropoffs).
- **Lower East Side (ID 148):** Στις **18:00** καταγράφει πλεονάζουσες αφίξεις της τάξης του **-2.094** (1.791 pickups έναντι 3.885 dropoffs).

Ερμηνεία για Οδηγούς: Οι περιοχές αυτές (East Village, West Village, Lower East Side) αποτελούν οικιστικές ζώνες αλλά και δημοφιλή κέντρα εστίασης και νυχτερινής διασκέδασης στο Μανχάταν. Τις απογευματινές ώρες, ο κόσμος που σχολούσε από τα γραφεία του Midtown μετακινείται μαζικά προς αυτές τις γειτονιές για επιστροφή ή έξοδο. Η οδηγία προς τους οδηγούς ταξί είναι σαφής: **να μην παραμένουν άσκοπα σε αυτές τις ζώνες μετά από ένα dropoff**, καθώς η πιθανότητα άμεσης επαναπαραλαβής είναι χαμηλή· πρέπει να επιστρέφουν άμεσα στους επιχειρηματικούς κόμβους.

```
maria@DESKTOP-Maria:~/bigdata-uth$ cat ~/final_results/q6_df_top_neg_net/part-*.csv
pickup_hour,location_id,pickups,dropoffs,activity,net_pickups,imbalance_ratio,abs_imbalance_ratio,LocationID,Borough,Zon
e,service_zone
18,79,4992,7908,12900,-2916,-0.22604651162790698,0.22604651162790698,79,Manhattan,East Village,Yellow Zone
18,249,5083,7335,12418,-2252,-0.18134965372845868,0.18134965372845868,249,Manhattan,West Village,Yellow Zone
17,79,3671,5895,9566,-2224,-0.232490068994355,0.232490068994355,79,Manhattan,East Village,Yellow Zone
18,148,1791,3885,5676,-2094,-0.3689217758985201,0.3689217758985201,148,Manhattan,Lower East Side,Yellow Zone
17,48,5700,7752,13452,-2052,-0.15254237288135594,0.15254237288135594,48,Manhattan,Clinton East,Yellow Zone
17,249,3917,5813,9730,-1896,-0.1948612538540596,0.1948612538540596,249,Manhattan,West Village,Yellow Zone
18,263,5446,7287,12733,-1841,-0.14458493677784497,0.14458493677784497,263,Manhattan,Yorkville West,Yellow Zone
16,48,4968,6762,11730,-1794,-0.15294117647058825,0.15294117647058825,48,Manhattan,Clinton East,Yellow Zone
18,230,10633,12397,23030,-1764,-0.07659574468085106,0.07659574468085106,230,Manhattan,Times Sq/Theatre District,Yellow Z
one
18,48,6619,8360,14979,-1741,-0.11622938780960011,0.11622938780960011,48,Manhattan,Clinton East,Yellow Zone
16,229,3703,5378,9081,-1675,-0.18445105164629447,0.18445105164629447,229,Manhattan,Sutton Place/Turtle Bay North,Yellow
Zone
18,239,8141,9730,17871,-1589,-0.08891500195848022,0.08891500195848022,239,Manhattan,Upper West Side South,Yellow Zone
18,50,1558,3106,4664,-1548,-0.3319039451114923,0.3319039451114923,50,Manhattan,Clinton West,Yellow Zone
17,263,5038,6582,11620,-1544,-0.13287435456110155,0.13287435456110155,263,Manhattan,Yorkville West,Yellow Zone
```

6(γ) Πείραμα Κλιμάκωσης (OD-Halves) - Πίνακας Benchmarks

Για τη μελέτη της συμπεριφοράς του Spark σε ένα βαρύ ερώτημα που εκτελεί inner join ανάμεσα σε δύο μεγάλα dataframes (διαχωρισμός του έτους 2024 σε H1 και H2), προκύπτουν οι εξής πραγματικοί χρόνοι εκτέλεσης ανά υπολογιστική διάταξη

Πίνακας Benchmark Κλιμάκωσης (Scalability & Analytics Matrix)

Βάσει των καταγραφών του Spark History Server (όπως προκύπτει από τα logs εκτέλεσης και το q6_scaling.png), προκύπτουν τα εξής στοιχεία:

Διάταξη	**Πόροι (Executors × Cores × Mem)**	**Χρόνος (Cold Run)**	**Χρόνος (Warm Run)**	**Επιλογή Τελεστή Catalyst**
Διάταξη Α	2 Executors $\times$ 1 Core $\times$ 1GB	88.66 s	78.43 s	`SortMergeJoin` (Shuffle-based)

Διάταξη B	4 Executors $\times$ 1 Core $\times$ 1GB	59.25 s	45.41 s	`SortMergeJoin` (Adaptive On)
Διάταξη C	4 Executors $\times$ 2 Cores $\times$ 1GB	46.88 s	37.58 s	`SortMergeJoin` (WholeStageCodegen)

Ανάλυση Σχεδίου Εκτέλεσης (EXPLAIN FORMATTED):

- **Τελεστής Συνένωσης:** Ο Catalyst Optimizer απέκλεισε τη στρατηγική Broadcast Join, καθώς και τα δύο υποσύνολα (H1 και H2) για ολόκληρο το 2024 είναι πολύ μεγάλα για να χωρέσουν στη μνήμη ενός executor. Επιλέχθηκε ο τελεστής **`SortMergeJoin`**.
- **Στάδια (Stages) και Δίκτυο:** Η εκτέλεση απαιτεί εκτεταμένο **Network Shuffle** προκειμένου να ανακατανεμηθούν και να ταξινομηθούν τα δεδομένα με βάση τα σύνθετα κλειδιά ομαδοποίησης (`pu_location_id`, `do_location_id`, `pickup_hour`).
- **Adaptive Execution:** Εμφανίζονται adaptive τελεστές (`AdaptiveSparkPlan`), οι οποίοι κατά το runtime συνενώνουν μικρά partitions (coalescing shuffle partitions) για να μειώσουν το overhead των tasks.

6(ε) Συμπεράσματα Ερωτήματος Q6

Παράγραφος 1: Επιχειρησιακό και Αναλυτικό Συμπέρασμα

Η χωρική ανάλυση του συνόλου δεδομένων του 2024 αναδεικνύει ότι η μεγαλύτερη απόλυτη ανισορροπία (`net_pickups`) εντοπίζεται στα αεροδρόμια **JFK** και **LaGuardia**, καθώς και στο **Midtown Center**. Η ώρα με τη μεγαλύτερη χωρική διαφορά επιβιβάσεων/αποβιβάσεων είναι το διάστημα **17:00 - 18:00 (απογευματινή αιχμή)**. Κατά την ώρα αυτή, η ζήτηση για αναχωρήσεις συγκεντρώνεται επιθετικά στις επιχειρηματικές ζώνες, προκαλώντας μαζική ροή οχημάτων προς τις οικιστικές περιοχές και τα κέντρα αναψυχής του Μανχάταν (East/West Village), τα οποία συσσωρεύουν μεγάλο πλεόνασμα αποβιβάσεων. Αντίθετα, οι απομακρυσμένες περιοχές της περιφέρειας εμφανίζουν ακραίο αρνητικό `imbalance_ratio`, υπογραμμίζοντας το φαινόμενο του άδειου οχήματος (deadheading) κατά την επιστροφή.

Παράγραφος 2: Τεχνικό Συμπέρασμα Κλιμακωσιμότητας

Το πείραμα κλιμάκωσης (OD-Halves) κατέδειξε ότι η αύξηση των υπολογιστικών πόρων επιφέρει σημαντική βελτίωση, αλλά με φθίνουσα απόδοση (μη γραμμική επιτάχυνση). Μετάβαση από τη Διάταξη A στη B (διπλασιάζοντας τους executors) μείωσε τον "warm" χρόνο κατά ~42% (από 78.43s σε 45.41s), ενώ η Διάταξη C (Vertical Scaling με 2 Cores) πέτυχε την

καλύτερη επίδοση στα 37.58s. Η διαφορά μεταξύ Cold και Warm εκτέλεσης οφείλεται στο instantiation των executors, το TCP Connection pre-warming και το JVM JIT compilation κατά την πρώτη εκτέλεση. Στο συγκεκριμένο query, οι φάσεις του τοπικού Map-side aggregation παραλληλίζονται πλήρως, ωστόσο το **Network Shuffle I/O** για το SortMergeJoin αποτελεί το κυρίαρχο σειριακό bottleneck (Network Contention), το οποίο περιορίζει την ιδανική γραμμική κλιμάκωση του cluster.

- **Είσοδος (Input):** `yellow_tripdata_2024.parquet`
- **Έξοδος (Output):** `/user/marpolymerou/project2026/results/q6_output/`
- **Spark Application IDs:** `app-20260518031540-0060` (4 Executors) και `app-20260518032212-0064`

Εκπλήξεις και Αναστοχασμός

Η ενασχόληση με το συγκεκριμένο project κατέστησε σαφές ότι η θεωρητική σχεδίαση ενός αλγορίθμου διαφέρει σημαντικά από την εκτέλεσή του σε ένα πραγματικά κατανεμημένο περιβάλλον (cluster).

Εκπλήξεις: Η μεγαλύτερη έκπληξη κατά τη διάρκεια των δοκιμών ήταν η τεράστια αρνητική επίδραση των Custom User Defined Functions (UDFs) στο ερώτημα Q2. Ενώ θεωρητικά μια custom συνάρτηση σε Python προσφέρει ευελιξία, στην πράξη ανάγκασε το Spark να κάνει διαδοχικά serialize και deserialize τα δεδομένα από τη JVM στην Python, ακυρώνοντας τις βελτιστοποιήσεις του Catalyst Optimizer και εκτινάσσοντας τον χρόνο εκτέλεσης.

Διάψευση Υποθέσεων EDA: Κατά το στάδιο της Εξερευνητικής Ανάλυσης Δεδομένων (EDA), η αρχική υπόθεση ήταν ότι οι ζώνες με τα υψηλότερα έσοδα και τη μεγαλύτερη κίνηση (όπως το κέντρο της πόλης) θα παρουσίαζαν και τη μεγαλύτερη σταθερότητα. Ωστόσο, η ανάλυση του Absolute Imbalance στο Q6 διέψευσε αυτή την προσδοκία, αποκαλύπτοντας ότι απομακρυσμένες ζώνες (π.χ. κοντά σε αεροδρόμια ή περιφερειακούς κόμβους) εμφάνιζαν βίαιες και ακραίες ανισορροπίες ροών σε πολύ συγκεκριμένες ώρες της ημέρας.

Αναστοχασμός: Αν ξεκινούσα το project από την αρχή, η πρώτη προτεραιότητα θα ήταν η εξ αρχής μετατροπή όλων των δεδομένων σε Parquet format και η άμεση σχεδίαση μιας ενιαίας δομής εκτέλεσης. Στα πρώτα στάδια, αναλώθηκε πολύτιμος χρόνος κάνοντας ξεχωριστές προσπάθειες και δοκιμές για κάθε API (RDD, DataFrames, SQL) σε μεμονωμένα, αρχεία κώδικα όπως προέβλεπε η εργασία. Ωστόσο, λόγο του όγκου των logs και το overhead της συνεχούς υποβολής πολλαπλών jobs στο cluster.
οδηγήθηκα στη συνέχεια στην ανάγκη να ενοποιήσω τα ερωτήματα Q2, Q3, Q4 και Q5 σε **"All-in-one" (τελικά συγκεντρωτικά) scripts**. Η στρατηγική αυτή επιλέχθηκε τελικά για να διασφαλίσω ότι η ανάγνωση των δεδομένων, οι μετασχηματισμοί και οι συγκρίσεις των APIs θα γίνονται κάτω από το ίδιο SparkSession, αποφεύγοντας τις αποτυχημένες υποβολές και μειώνοντας δραματικά τον χρόνο που χανόταν στις ουρές του YARN. Αν ήξερα απο την αρχή ότι δεν θα καταφέρω να τα τρέξω όπως πρέπει, θα είχα γλιτώσει τον κατακερματισμό των αρχικών δοκιμών και θα είχα διαθέσει αυτόν τον χρόνο στην περαιτέρω παραμετροποίηση των partitions.

Ο τελικός Πίνακας Κάλυψης

Απαιτούμενο Παραδοτέο	**Κάλυψη (Ναι / Όχι)**	**Τοποθεσία στο Repository /**
Ερώτημα Q1 (Spark RDD / DataFrame / SQL & Σύγκριση)	**Ναι**	Κώδικας: `jobs/q1_df.py` /
Ερώτημα Q2 (Built-in vs Custom UDF)	**Ναι**	Κώδικας: `jobs/q2_final_all.py` /
Ερώτημα Q3 (Partition Pruning Πείραμα)	**Ναι**	Κώδικας: `jobs/q3_final_all.py` /
Ερώτημα Q4 (Ανάλυση πληρωμών / tip rates)	**Ναι**	Κώδικας: `jobs/q4_all.py` /
Ερώτημα Q5 (Broadcast Join vs Sort-Merge Join)	**Ναι**	Κώδικας: `jobs/q5_all.py` /
Ερώτημα Q6 (Μελέτη Κλιμάκωσης & 3 Διατάξεις)	**Ναι**	Κώδικας: `jobs/q6_df.py` & `run_q6_repeated.sh` /
Συγκριτικοί Πίνακες Χρόνων Εκτέλεσης	**Ναι**	Συγκεντρωτικός Πίνακας στην Αναφορά
Διαγράμματα / Plots (Ζήτηση, Έσοδα, Ροές, Scaling)	**Ναι**	Ενσωματωμένα στην Αναφορά & Φάκελος `results/`
Script Καθολικής Αυτοματοποίησης (`run_all.sh`)	**Ναι**	Φάκελος `scripts/run_all.sh`
Οδηγός Αναπαραγωγής Αποτελεσμάτων (`RUNBOOK.md`)	**Ναι**	Ρίζα του Αποθετηρίου (`RUNBOOK.md`)

Δήλωση Χρήσης LLM (`LLM_USAGE.md`)	**Ναι**	Αρχείο `LLM_USAGE.md` & Αναφορά

Χρήση LLM

Στο πλαίσιο της παρούσας εργασίας, έγινε χρήση εργαλείων Τεχνητής Νοημοσύνης (Gemini & Claude) αποκλειστικά σε ρόλο βοηθητικού εργαλείου για τους εξής σκοπούς:

- Υποστήριξη στη σωστή δομή των φακέλων του αποθετηρίου σύμφωνα με τις απαιτήσεις της εκφώνησης.
- Βοήθεια στην αποκωδικοποίηση και ερμηνεία πολύπλοκων μηνυμάτων λάθους (stack traces) του Spark και της Java κατά τις δοκιμές στο cluster, καθώς και στην επίλυση συντακτικών αστοχιών.
- Παροχή κατευθύνσεων σε συγκεκριμένα σημεία όπου υπήρχαν δυσκολίες στη βέλτιστη υλοποίηση των ερωτημάτων (όπως η σωστή δομή των εναλλακτικών joins).
- Δημιουργία του αρχικού σκελετού (boilerplate) για τις βιβλιοθήκες Matplotlib και Seaborn, διευκολύνοντας τη γρήγορη και καθαρή σχεδίαση των γραφημάτων (ρύθμιση αξόνων, χρωματικές παλέτες χρόνων εκτέλεσης).
- Βοήθεια στη μορφοποίηση, σύνταξη και οργάνωση των τελικών αρχείων τεκμηρίωσης Markdown (README.md, RUNBOOK.md) καθώς και της δομής του κειμένου της αναφοράς ώστε να παρουσιάζονται τα ευρήματα με σαφή τρόπο.